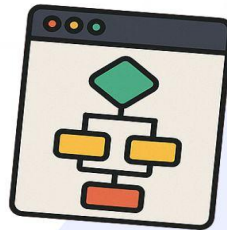
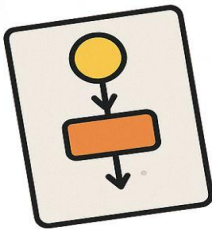




ALGORITMA & PEMROGRAMAN



I Gde Agung Sri Sidhimantra, S.Kom., M.Kom.
Binti Kholifah, S.Kom., M.Tr.Kom.
M Adamu Islam Mashuri, S.Tr.T., M.Tr.Kom.
Faris Abdi El Hakim, S.Kom., M.Tr.Kom.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya buku ini yang berjudul *Algoritma dan Pemrograman Python* dapat diselesaikan dengan baik. Buku ini disusun sebagai panduan pembelajaran dasar-dasar algoritma dan pemrograman menggunakan bahasa Python, yang diharapkan dapat membantu mahasiswa memahami konsep pemrograman secara sistematis dan aplikatif.

Isi buku mencakup pengenalan bahasa Python, penggunaan variabel dan operator, struktur percabangan, perulangan, fungsi, hingga pengolahan file dan proyek mini sebagai aplikasi nyata. Dengan pendekatan yang disusun secara bertahap, buku ini dirancang untuk memudahkan pembaca, khususnya mahasiswa pemula, dalam memahami logika pemrograman dan mengimplementasikannya melalui bahasa Python yang sederhana namun sangat powerful.

Penulis menyadari bahwa penyusunan buku ini tidak lepas dari dukungan dan kontribusi berbagai pihak, baik dari rekan dosen, mahasiswa, maupun institusi yang memberikan masukan dan semangat selama proses penulisan. Oleh karena itu, penulis mengucapkan terima kasih yang sebesar-besarnya kepada semua pihak yang telah membantu dalam penyusunan buku ini.

Harapan penulis, buku ini dapat menjadi sumber belajar yang bermanfaat dalam mendukung proses pembelajaran algoritma dan pemrograman serta mendorong lahirnya pemrogram muda yang kreatif dan logis. Kritik dan saran yang membangun sangat penulis harapkan untuk penyempurnaan buku ini di masa yang akan datang.

Akhir kata, semoga buku ini dapat memberikan kontribusi positif bagi pengembangan ilmu pengetahuan dan teknologi, khususnya di bidang informatika dan pemrograman.

Surabaya, Juli 2025

Tim Dosen Algoritma Pemrograman

Sarjana Terapan Manajemen Informatika - UNESA

DAFTAR ISI

KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I PENGENALAN BAHASA PYTHON	1
BAB II VARIABEL, OPERATOR, INPUT	17
BAB III CONDITIONAL DAN LOGICAL OPERATOR	37
BAB IV LOOPING	45
BAB V FUNGSI	53
BAB VI FILE	65
BAB VII MINI PROJECT	75

BAB I

PENGENALAN BAHASA PYTHON

Tujuan Pembelajaran :

1. Mahasiswa mengetahui tentang Python dan kelebihanannya
2. Mahasiswa mampu mengunduh dan menginstal versi terbaru Python dari situs resmi Python beserta IDE
3. Mahasiswa mengerti apa itu pseudocode dan bagaimana menulis pseudocode yang jelas dan terstruktur untuk menggambarkan algoritma
4. Mahasiswa mampu membuat flowchart untuk menggambarkan algoritma secara visual sehingga memudahkan pemahaman dan komunikasi tentang proses yang dijelaskan
5. Mahasiswa memahami cara menulis, menjalankan, dan menyimpan kode Python menggunakan IDE

1.1 Sejarah

Pada tahun 1982 Guido van Rossum, pencipta Python memulai bekerja di CWI yang merupakan sebuah lembaga penelitian nasional Belanda. Dia bergabung dengan tim yang sedang merancang bahasa pemrograman baru bernama ABC yang digunakan untuk pengajaran dan pembuatan prototipe. Kesederhanaan ABC membuatnya ideal untuk pemula tetapi bahasa tersebut kekurangan fitur yang diperlukan untuk menulis program yang lebih canggih.

Beberapa tahun kemudian, van Rossum bergabung dengan tim berbeda di CWI yang bekerja pada bidang sistem

operasi. Tim tersebut membutuhkan cara yang lebih mudah untuk menulis program pemantauan komputer dan menganalisis data. Bahasa-bahasa yang umum digunakan pada tahun 1980-an (dan bahkan hingga kini) dianggap sulit untuk program semacam ini. Van Rossum membayangkan sebuah bahasa baru yang memiliki sintaks sederhana seperti ABC tetapi juga menyediakan fitur-fitur canggih yang dibutuhkan para profesional.

Awalnya, van Rossum mulai mengerjakan bahasa baru ini sebagai hobi di waktu luangnya. Dia menamai bahasa tersebut Python karena dia adalah penggemar grup komedi Inggris, Monty Python. Selama setahun berikutnya, ia dan rekan-rekannya berhasil menggunakan Python untuk berbagai pekerjaan nyata. Van Rossum akhirnya memutuskan untuk membagikan Python kepada komunitas pemrograman yang lebih luas secara online. Dia membagikan seluruh kode sumber Python secara gratis sehingga siapa pun dapat menulis dan menjalankan program Python.

Pada tahun 1991 Python rilis pertama dengan versi 0.9.0 enam tahun setelah C++ dan empat tahun sebelum Java. Keputusan van Rossum untuk membuat bahasa yang sederhana namun canggih, cocok untuk tugas sehari-hari, tersedia secara gratis secara online yang berkontribusi pada kesuksesan jangka panjang Python.

1.2 Kelebihan Python

Beberapa kelebihan dari Python :

1. Mudah dipelajari dan digunakan

Syntax Python sangat sederhana dan developer tidak perlu menulis banyak kode boilerplate (kode berulang yang tidak esensial) yang membuat kode Python lebih mudah dipahami dan dipelajari dibandingkan dengan bahasa pemrograman lainnya.

2. Banyak Library dan Tools

Dengan menggunakan Python mengerjakan project dapat diselesaikan dengan mudah karena memiliki banyak library yang dapat digunakan, contohnya terdapat library untuk pengembangan web yaitu django dan flask, analisis data yaitu numpy dan pandas, machine learning yaitu tensorflow dan keras, dan masih banyak library lain.

3. Produktivitas dan pengembangan cepat

Dengan sintaks Python yang sederhana serta banyak library yang tersedia untuk digunakan memungkinkan pengembangan lebih cepat.

4. Manajemen memory yang efisien

Python memiliki cara tersendiri dalam mengelola memori yang terkait dengan objek. Ketika sebuah objek dibuat di Python, memori dialokasikan secara dinamis untuk objek tersebut. Ketika objek sudah tidak digunakan, memori dikembalikan. Pengelolaan memori Python ini membuat program menjadi lebih efisien.

5. Bahasa Interpreted

Interpreted artinya kode dapat dieksekusi baris demi baris tanpa perlu dikompilasi yang memudahkan pengembangan dan debugging.

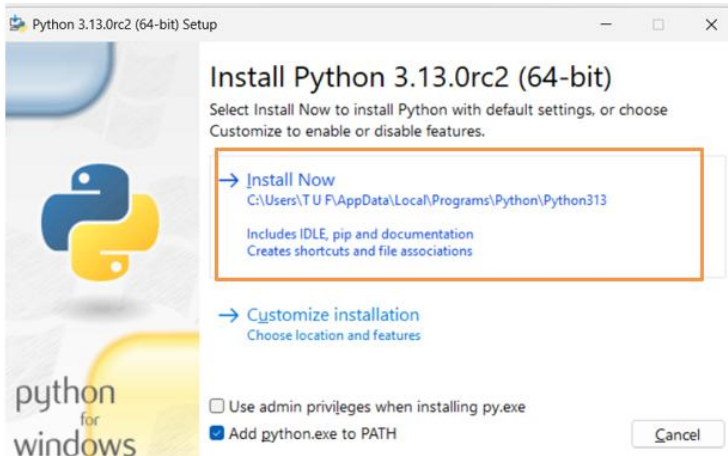
1.3 Instalasi Python

Berikut ini merupakan langkah-langkah proses instalasi Python :

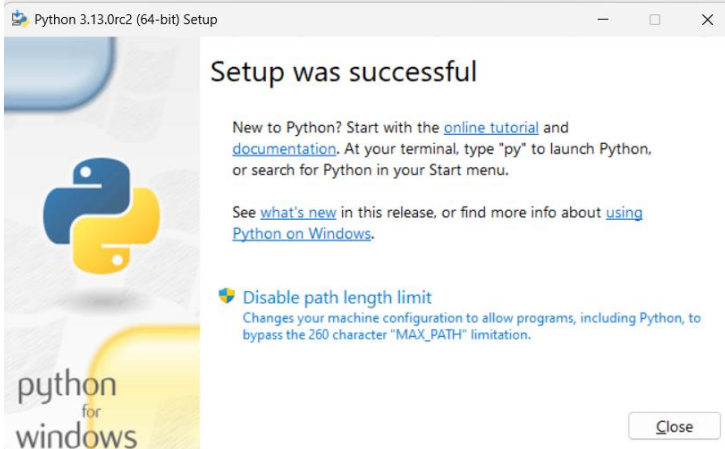
1. Buka browser dan kunjungi situs resmi Python di <https://www.python.org/downloads/>
2. Download Python versi 3 sesuai sistem operasi perangkat komputer Anda.



3. Buka file installer Python yang telah diunduh.
4. Centang Add python.exe to PATH untuk memudahkan penggunaan Python dari Command Prompt tanpa perlu mengatur PATH secara manual.
5. Klik tombol Install now untuk proses instalasi.



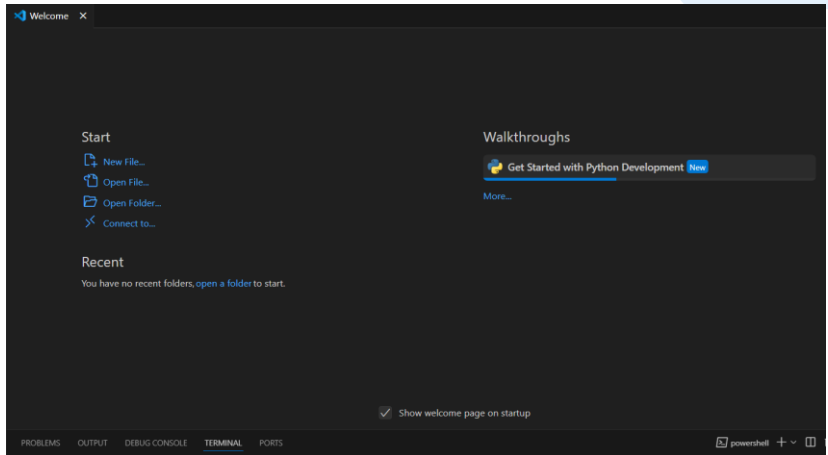
6. Setelah proses instalasi Python selesai maka akan muncul popup window seperti gambar dibawah.



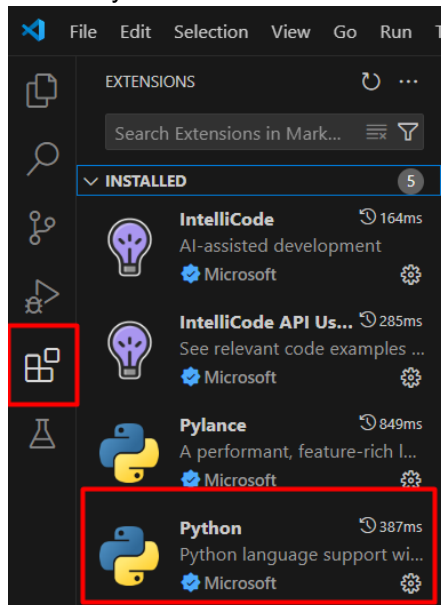
7. Download Visual Studio Code (VS Code) sebagai aplikasi editor kode pada alamat <https://code.visualstudio.com/download>. Download sesuai dengan sistem operasi anda.



8. Tampilan awal VS Studio Code setelah berhasil diinstall pada perangkat anda.



9. Pilih Extensions pada navbar sisi kiri, kemudian search, pilih dan install “Python” for VS Studio Code.



1.4 Algoritma

Algoritma adalah suatu langkah-langkah atau prosedur yang sistematis untuk menyelesaikan masalah atau mencapai tujuan tertentu. Dalam dunia pemrograman dan komputasi, algoritma digunakan untuk menggambarkan bagaimana suatu masalah dapat dipecahkan melalui serangkaian instruksi yang diikuti oleh mesin atau komputer. Secara umum algoritma terbagi menjadi 3 bagian yaitu deskriptif, pseudocode, dan flowchart.

1.4.1. Deskriptif

Algoritma deskriptif adalah cara menjelaskan langkah-langkah penyelesaian suatu masalah secara naratif (berupa kalimat). Bentuk ini tidak menggunakan simbol-simbol khusus ataupun format pemrograman, tetapi ditulis dalam bahasa manusia yang mudah dimengerti.

Penulisan algoritma secara deskriptif cocok digunakan pada tahap awal analisis masalah sebelum diterjemahkan menjadi bentuk pseudocode atau kode program. Pendekatan ini sangat membantu untuk membiasakan mahasiswa berpikir logis secara terstruktur.

Penjelasan langkah-langkah dalam menyelesaikan suatu masalah yang ditulis secara naratif atau dalam bentuk kalimat deskriptif biasa.

Karakteristik Algoritma Deskriptif

- Ditulis dalam bentuk kalimat atau poin-poin naratif.
- Tidak menggunakan aturan sintaks khusus.
- Mudah dibaca dan dipahami oleh siapa saja, termasuk yang belum belajar pemrograman.
- Cocok sebagai pengantar sebelum membuat pseudocode atau flowchart.

Contoh algoritma deskriptif untuk menentukan bilangan tersebut ganjil atau genap:

1. Mulai program.
2. Masukkan bilangan.
3. Hitung sisa pembagian bilangan dengan 2 menggunakan operator modulus (%).
4. Cek hasil sisa bilangan jika sisa sama dengan 0:
 - Jika ya, bilangan tersebut adalah **genap**.
 - Jika tidak, bilangan tersebut adalah **ganjil**.
5. Tampilkan hasilnya.
6. Akhiri program.

1.4.2. Pseudocode

Cara untuk menulis algoritma dalam bentuk notasi mirip kode program, tetapi tidak mengikuti sintaks dari bahasa pemrograman tertentu. Pseudocode dirancang untuk mudah dipahami oleh manusia, karena menggunakan struktur yang menyerupai kode tetapi tetap bisa ditangkap dengan jelas oleh orang yang tidak terlalu paham pemrograman.

Pseudocode menjembatani antara deskripsi naratif dan kode program sesungguhnya. Dengan pseudocode, kita dapat mendesain solusi terlebih dahulu sebelum menerjemahkannya ke dalam kode Python, C++, Java, atau bahasa lainnya.

Karakteristik Pseudocode:

- Menggunakan bahasa natural (biasanya bahasa Indonesia atau Inggris) yang disederhanakan.
- Tidak memperhatikan aturan sintaks pemrograman yang spesifik.
- Fokus pada langkah logis dan alur kontrol program (urutan, percabangan, pengulangan).

- Cocok untuk diskusi tim, perencanaan program, atau dokumentasi algoritma.

Contoh pseudocode untuk menentukan bilangan tersebut ganjil atau genap:

```
Mulai
Masukkan bilangan
sisa = bilangan % 2
Jika (bilangan mod 2 == 0) maka
    Cetak "Bilangan Genap"
Jika tidak
    Cetak "Bilangan Ganjil"
Selesai
```

1.4.3. Flowchart

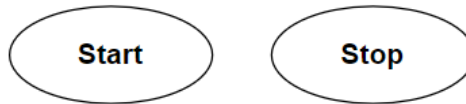
Flowchart adalah representasi dari langkah-langkah dalam suatu proses atau algoritma. Ini digunakan untuk merencanakan dan menggambarkan logika suatu program secara visual sebelum diimplementasikan dalam kode. Dengan kata lain, flowchart dapat digunakan untuk merancang algoritma yang kemudian bisa diterjemahkan ke dalam berbagai bahasa pemrograman. Flowchart menggunakan bentuk yang berbeda untuk menunjukkan jenis instruksi yang berbeda.

Manfaat Flowchart:

- Membantu memahami logika program secara menyeluruh.
- Menyediakan visualisasi yang jelas untuk setiap tahapan proses.
- Mempermudah diskusi dan dokumentasi program.
- Membantu mengidentifikasi kesalahan logika sejak awal perancangan.

Berikut ini merupakan beberapa simbol yang digunakan dalam flowchart :

1. Terminator



Simbol terminator digunakan untuk menunjukkan titik awal dan akhir dari suatu proses atau program.

2. Input/Output



Input / Output

Simbol ini digunakan untuk menunjukkan fungsi input/output dalam program. Jadi, jika ada input/output ke program akan ditunjukkan dalam flowchart dengan menggunakan simbol Input/Output.

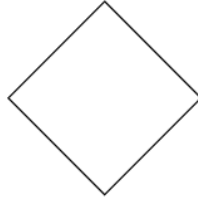
3. Process



Process

Digunakan untuk menunjukkan langkah-langkah dalam proses di mana data diproses atau tindakan dilakukan. Semua proses seperti perhitungan aritmatika, pemindahan data, atau operasi lain.

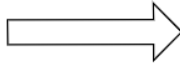
4. Decision



Decision

Decision dalam diagram alir (flowchart) digunakan untuk menunjukkan titik di mana sebuah keputusan harus diambil, dalam bentuk ya dan tidak.

5. Flowline



Flowline

Flowline adalah garis solid dengan ujung panah yang menunjukkan alur operasi dalam sebuah flowchart. Mereka memperlihatkan urutan tepat di mana instruksi atau langkah-langkah harus dijalankan. Flowlines membantu membimbing pembaca melalui proses dengan jelas, menunjukkan arah alur kerja dan hubungan antara berbagai simbol dalam diagram.

6. Connector

Dalam situasi di mana program yang cukup kompleks membuat diagram alir (flowchart) menjadi besar mengakibatkan garis aliran (flowlines) akan saling bersilangan di banyak tempat sehingga menyebabkan kebingungan dan sulit dipahami saat membaca flowchart atau terdapat proses yang terputus pada flowchart. Maka

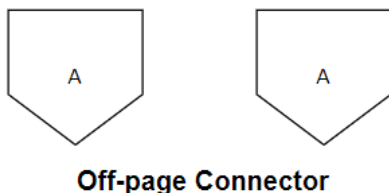
masalah tersebut dapat diatasi dengan connector dalam flowchart untuk menghubungkan bagian-bagian dari diagram yang terpisah.

- a. On-page Conector (dalam halaman)



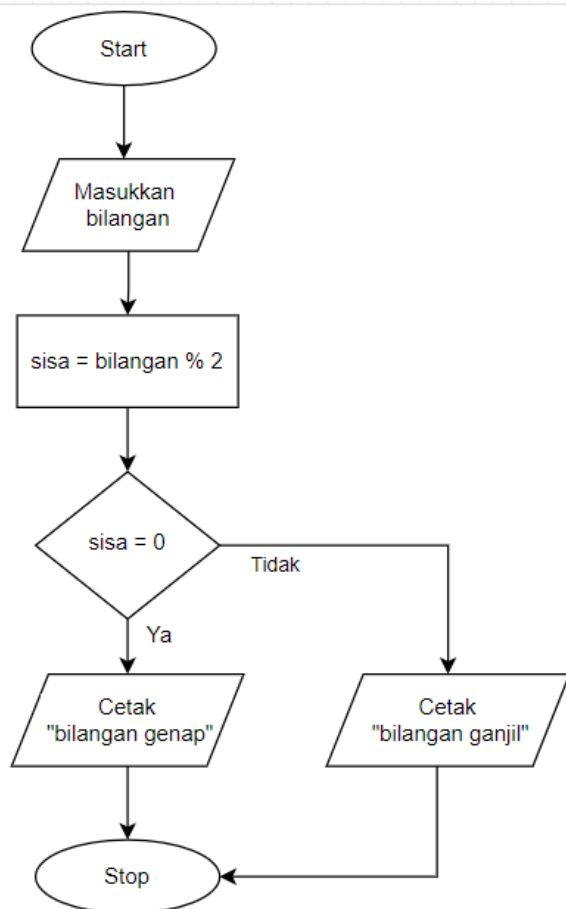
Digunakan dalam aliran diagram (flowchart) dalam satu halaman diberi tanda dengan huruf atau angka untuk memperjelas koneksi antar dua titik alur.

- b. Off-page Connector (antar halaman)



Digunakan dalam aliran diagram (flowchart) dalam antar halaman bahwa aliran proses dilanjutkan dalam halaman lain diberi tanda dengan huruf atau angka untuk memperjelas koneksi antar dua titik alur dalam halaman yang berbeda.

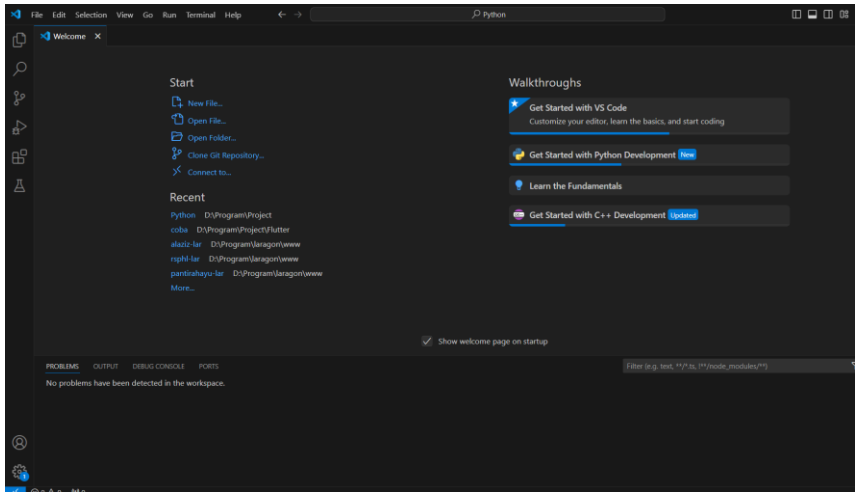
Contoh penggunaan flowchart menentukan bilangan genap atau ganjil sesuai dengan alur deskriptif dan pseudocode sebelumnya. :



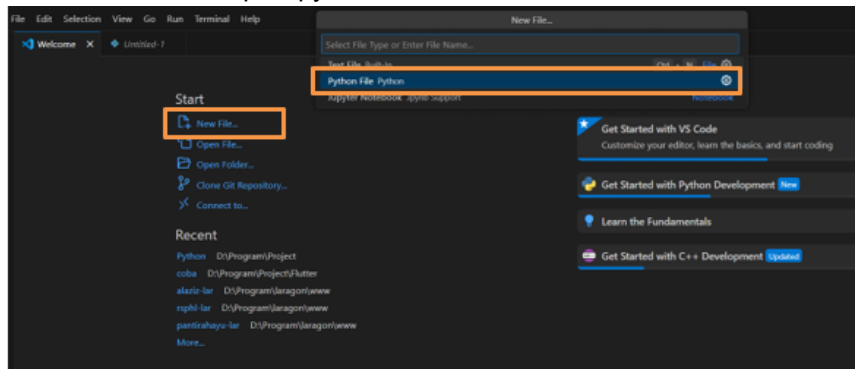
1.5 Menjalankan Python pada IDE

Berikut ini merupakan langkah-langkah menjalankan program Python pada Integrated Development Environment (IDE) yaitu Visual Studio Code :

1. Buka Visual Studio Code pada komputer anda.

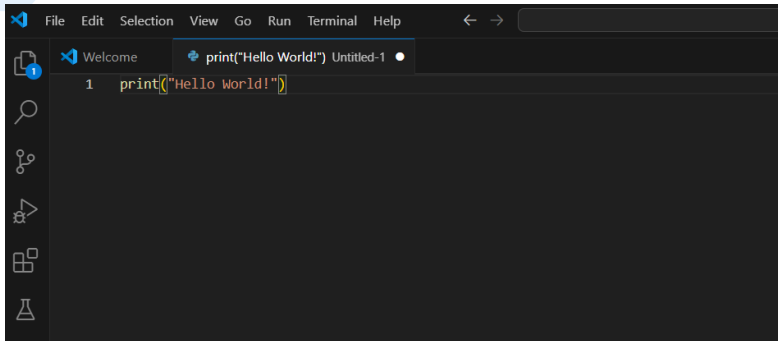


2. Klik new file and pilih python file.

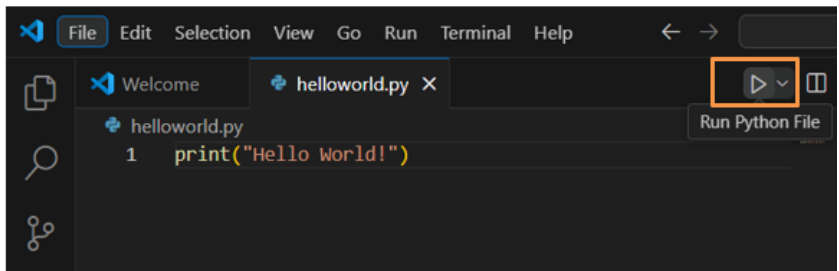


3. Kali ini kita akan membuat program yang menampilkan output Hello World!

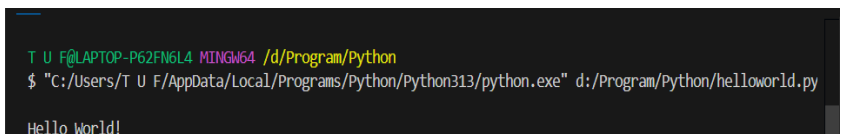
4. Tulis kode program yang akan dieksekusi dengan melakukan perintah `print("Hello World!")`.



5. Simpan file tersebut dengan menekan shortcut `ctrl + s`. Beri nama file kemudian klik save.
6. Eksekusi program dengan menekan tombol run python file sesuai dengan gambar dibawah ini.



7. Maka akan tampil output sesuai dengan kode yang sudah ditulis.



1.6 Tugas

1. Buatlah algoritma deskriptif dalam proses login sso unesa.
2. Buatlah pseudocode untuk menghitung luas lingkaran.
3. Buatlah flowchart yang meminta pengguna memasukkan usia mereka dan menampilkan pesan berdasarkan usia :
Jika usia kurang dari 18, tampilkan "Anda masih remaja."
Jika usia antara 18 dan 60, tampilkan "Anda adalah orang dewasa."
Jika usia lebih dari 60, tampilkan "Anda adalah lansia."

BAB II

VARIABEL, OPERATOR, INPUT

Tujuan Pembelajaran :

1. Mahasiswa mengetahui penulisan kode Python
2. Mahasiswa memahami apa itu variabel, aturan penulisan variabel, serta penggunaan variabel
3. Mahasiswa mengetahui berbagai tipe data dasar
4. Mahasiswa mempelajari bagaimana menggunakan fungsi input
5. Mahasiswa memahami berbagai jenis operator dalam bahasa pemrograman
6. Mahasiswa mampu menggunakan interpreter pada python

2.1 Struktur Penulisan Kode

Dalam penulisan kode Python terdapat berbagai aturan untuk menjaga keterbacaan dan fungsionalitas program diantaranya :

- a. Komentar digunakan untuk menjelaskan bagian-bagian kode tertentu dengan menggunakan tanda # diawal dan diakhir untuk 1 baris komentar atau menggunakan ''' atau """ diawal dan diakhir komentar untuk banyak baris.

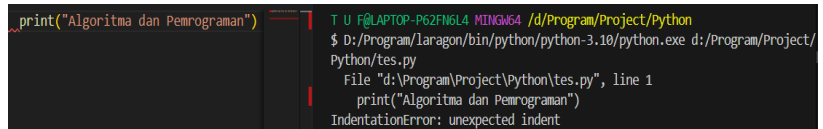
```
#tes#  
print("Algoritma dan Pemrograman")
```

```
'''tes komentar saja  
menggunakan 2 baris komentar
```

```
'''
print("Algoritma dan Pemrograman")
```

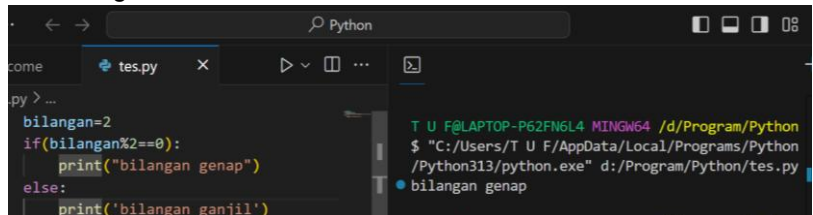
- b. Indentasi merupakan aturan spasi atau tab di awal suatu kode untuk menandakan struktur kode dalam program. Berikut ini merupakan contoh penggunaan indentasi dalam Python :

Contoh indentasi yang salah (yang seharusnya tanpa spasi atau tab pada awal fungsi print) terlihat terdapat error yang saat dijalankan:



```
print("Algoritma dan Pemrograman")
T U F@LAPTOP-P62FN6L4 MINGW64 /d:/Program/Project/Python
$ D:/Program/laragon/bin/python/python-3.10/python.exe d:/Program/Project/
Python/tes.py
File "d:/Program/Project/Python/tes.py", line 1
  print("Algoritma dan Pemrograman")
  IndentationError: unexpected indent
```

Berbeda dengan aturan indentasi harus menggunakan spasi atau tab setelah penggunaan percabangan, perulangan, serta fungsi:



```
bilangan=2
if(bilangan%2==0):
    print("bilangan genap")
else:
    print('bilangan ganjil')
T U F@LAPTOP-P62FN6L4 MINGW64 /d:/Program/Python
$ "C:/Users/T U F/AppData/Local/Programs/Python
/Python313/python.exe" d:/Program/Python/tes.py
bilangan genap
```

- c. Import merupakan fungsi memanggil atau mengakses library yang digunakan dalam program. Penulisan import dilakukan pada baris awal kode sebelum kode lain. Contoh penggunaan import dalam Python:

```
import math
print(math.sqrt(25))
# Output: 5.0
```

2.2 Variabel

Variabel adalah tempat penyimpanan pada memori yang dapat diubah pada suatu program.

2.2.1 Penamaan variabel

Terdapat beberapa aturan dalam penamaan variabel dalam penulisan kode dalam program yaitu:

- Penamaan variabel harus jelas sesuai dengan isi variabel tersebut. contoh : nama_mhs, jangan menggunakan nm_mhs.
- Nama variabel tidak boleh dimulai dengan angka.
- Tidak boleh memakai karakter khusus seperti spasi,!,@,# dan lain-lain.
- Nama variabel bersifat case-sensitive yang artinya variabel, Variabel, dan VARIABEL dianggap sebagai nama yang berbeda.
- Konsisten dengan gaya penulisan dalam nama variabel dengan memilih satu gaya penulisan seperti camel case (namaMHS) atau snake case (nama_mhs).

2.2.2 Mendeklarasikan dan inisiasi variabel

Deklarasi variabel adalah mendefinisikan atau membuat variabel dengan memberitahukan kepada program bahwa variabel akan digunakan dan inisiasi variabel atau memberikan nilai kepada variabel.

```
matkul = "AlgoritmadanPemrograman"  
print(matkul)
```

Pada program diatas telah memberikan nilai Algoritmadan Pemrograman ke dalam variabel matkul.

2.2.3 Mengubah Nilai dari Variabel

Nilai dari variabel dapat diubah dengan melakukan inisiasi nilai baru dengan variabel yang sudah ada.

```
matkul = "AlgoritmadanPemrograman"  
print(matkul)  
matkul = "Algoritma dan Pemrograman"  
print(matkul)
```

Pada program diatas telah memberikan nilai AlgoritmadanPemrograman ke dalam variabel matkul. Kemudian mengubah nilai pada variabel matkul dengan nilai baru yaitu Algoritma dan Pemrograman.

2.3 Tipe Data Dasar

Dalam variabel terdapat berbagai macam jenis nilai atau data yang disimpan. Saat Anda membuat variabel dan memberinya nilai, Python secara otomatis mengasosiasikan variabel tersebut dengan tipe data yang sesuai dengan nilai yang diberikan.

2.3.1. String

String adalah tipe data yang menyimpan teks. Ditandai oleh tanda kutip tunggal (') atau ganda (").

Contoh:



```
nama = 'Adi'  
pesan = "Selamat pagi"
```

String dapat dioperasikan menggunakan operator seperti + (penggabungan) dan * (pengulangan):

```
String.py

print("Halo " + "Dunia")    # Output: Halo Dunia
print("Ha" * 3)             # Output: HaHaHa
```

Python juga menyediakan berbagai metode string yang memudahkan pengolahan teks. Dua metode yang sangat sering digunakan adalah upper() dan lower().

- upper(): Mengubah semua huruf menjadi huruf besar (kapital). Metode upper() akan mengembalikan versi huruf kapital dari seluruh karakter alfabet dalam string.

```
Latihan.py

teks = "python"
print(teks.upper())    # Output: PYTHON
```

- lower(): Mengubah semua huruf menjadi huruf kecil. Metode lower() akan mengembalikan versi huruf kecil dari seluruh karakter alfabet dalam string.

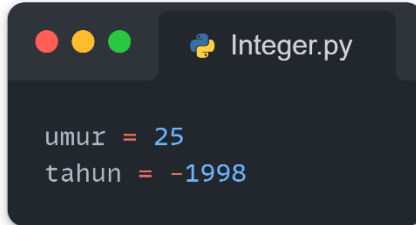
```
Latihan.py

teks = "PEMROGRAMAN"
print(teks.lower())    # Output: pemrograman
```


2.3.2. Integer

Integer adalah tipe data yang menyimpan bilangan bulat atau integer. Integer dapat berupa bilangan positif, nol, maupun bilangan negatif.

Contoh:



```
umur = 25
tahun = -1998
```

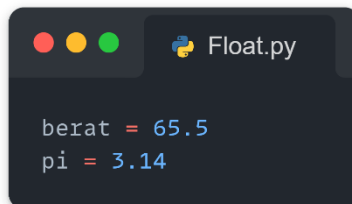
Dalam contoh diatas:

- Umur adalah integer positif
- Tahun adalah integer negatif

2.3.3. Float

Float adalah tipe data yang menyimpan bilangan desimal atau floating point. Float sangat penting ketika kita membutuhkan hasil perhitungan yang mengandung angka di belakang koma, misalnya dalam pengukuran, persentase, atau rumus matematika.

Contoh:



```
berat = 65.5
pi = 3.14
```

Berat dan pi merupakan bilangan bertipe float karena mengandung bilangan desimal.

2.3.4. Boolean

Tipe data yang menyimpan kondisi dalam bentuk True untuk nilai benar dan False untuk nilai salah. Contoh penggunaan tipe data dalam bahasa Python :

```
nama = "faris"
#Variabel nama berisi teks maka tipe datanya
string
angka = 10
#Variabel angka maka tipe datanya integer
phi = 3.14
#Variabel phi berisi nilai desimal maka tipe
datanya float
masuk = True
#Variabel phi berisi nilai boolean True
```

2.4 Keyword

Keyword merupakan kata khusus yang digunakan dalam sintaks bahasa pemrograman. Contohnya : True, False, as, and, or, not, if , elif, else, import, for, def, dan masih banyak yang lain.

```
import math as mtk
print(mtk.pow(5,2))
# Output: 25.0
```

Pada contoh program diatas menggunakan keyword import untuk memanggil operasi math dan menggunakan keyword as atau alias untuk mengganti math menjadi nama baru yaitu mtk.

2.5 Input

Input adalah masukan dari pengguna setelah menekan enter selama eksekusi program. Dalam pemrograman Python, kita sering membutuhkan input (masukan) dari pengguna agar program menjadi interaktif. Python menyediakan fungsi `input()` untuk menerima data dari pengguna saat program dijalankan.

Fungsi `input()` akan menampilkan pesan di layar, lalu menunggu pengguna mengetik sesuatu dan menekan Enter. Nilai yang dimasukkan akan selalu bertipe string (teks), meskipun yang dimasukkan adalah angka.

Contoh program menggunakan input dari pengguna:

```
print("Masukkan nama anda: ")
nama = input()
print("Halo :) "+nama)
```

2.6 Operator

Operator adalah simbol atau karakter khusus yang digunakan untuk melakukan operasi terhadap variabel dan nilai. Operator membantu pengguna melakukan operasi tertentu seperti penjumlahan, pengurangan, dan sebagainya. Python memiliki berbagai jenis operator yang memungkinkan kita melakukan perhitungan aritmatika, perbandingan, logika, penugasan, dan lainnya, contohnya sebagai berikut:

2.6.1. Operator Aritmatika

Operator aritmatika digunakan dalam melakukan operasi matematika, seperti penjumlahan, pengurangan, perkalian, pembagian, dll. Berikut ini merupakan operator aritmatika yang sering digunakan:

Operator	Keterangan
+	Penambahan
-	Pengurangan
/	Pembagian
*	Perkalian
**	Eksponen
%	Modulus

Contoh penggunaan operator aritmatika:

```
angka1 = 10
angka2 = 2
hasil = angka1 % 2
print(hasil)
```

Penjelasan program:

- `angka1 = 10`
Mendeklarasikan variabel `angka1` dan memberikan nilai 10.
- `angka2 = 2`
Mendeklarasikan variabel `angka2` dan memberikan nilai 2.
(Catatan: dalam kode ini, `angka2` tidak dipakai, jadi bisa dihapus atau digunakan untuk memperjelas maksud perhitungan)
- `hasil = angka1 % 2`
Melakukan operasi modulus (%) yaitu sisa pembagian `angka1` dibagi 2.
 $10 \% 2 = 0$, karena 10 habis dibagi 2. Nilai 0 disimpan ke dalam variabel `hasil`.
- `print(hasil)`
Menampilkan nilai dari variabel `hasil` ke layar.
Output: 0

2.6.2. Operator Penugasan

Operator penugasan digunakan untuk mengisi nilai pada suatu variabel.

Operator	Keterangan
=	Menandakan pemberian nilai variabel sebelah kiri dengan nilai sebelah kanan
+=	Menambah nilai variabel sebelah kiri dengan nilai di sebelah kanan
-=	Mengurangi nilai variabel sebelah kiri dengan nilai di sebelah kanan
*=	Mengalikan nilai variabel sebelah kiri dengan nilai di sebelah kanan
**=	Memangkatkan nilai variabel sebelah kiri dengan nilai di sebelah kanan
%=	Penggunaan modulus nilai variabel sebelah kiri dengan nilai di sebelah kanan

Contoh penggunaan operator penugasan:

```
angka1 = 10
angka1 += 2
print(angka1)
```

Penjelasan Program:

- `angka1 = 10`
Menefinisikan variabel `angka1` dan memberikan nilai awal **10**.
- `angka1 += 2`
Merupakan **operator penugasan gabungan** (`+=`), yang artinya:
`angka1 = angka1 + 2`
Jadi, nilai `angka1` ditambah 2 → menjadi `10 + 2 = 12`.

- `print(angka1)`
Menampilkan nilai `angka1` ke layar.
Karena `angka1` telah diubah menjadi 12, maka output program adalah "12".

2.6.3. Operator Pembandingan

Operator pembandingan digunakan untuk membandingkan dua nilai dalam pernyataan kondisional dan menghasilkan nilai boolean `true` atau `false`.

Operator	Keterangan
<code>==</code>	Sama dengan
<code>!=</code>	Tidak sama dengan
<code>></code>	Lebih besar
<code><</code>	Lebih kecil
<code>>=</code>	Lebih besar atau sama dengan
<code><=</code>	Lebih kecil atau sama dengan

Contoh penggunaan operator pembandingan:

```
# Mendefinisikan beberapa variable
a = 10
b = 20

# Menggunakan operator perbandingan dan
mencetak hasilnya
print("a == b:", a == b)
# Memeriksa apakah a sama dengan b
print("a < b:", a < b)
# Memeriksa apakah a kurang dari b
print("a > b:", a > b)
# Memeriksa apakah a lebih dari b
```

Penjelasan Program:

Program di atas merupakan contoh penggunaan operator perbandingan (relasional) dalam Python. Pertama, didefinisikan dua variabel yaitu a dengan nilai 10 dan b dengan nilai 20. Kemudian, program mencetak hasil dari beberapa ekspresi perbandingan antara a dan b. Ekspresi $a == b$ akan menghasilkan False karena nilai a tidak sama dengan b. Selanjutnya, ekspresi $a < b$ akan menghasilkan True karena 10 memang lebih kecil dari 20. Terakhir, ekspresi $a > b$ akan menghasilkan False karena a tidak lebih besar dari b. Program ini menunjukkan bahwa operator perbandingan digunakan untuk membandingkan dua nilai dan menghasilkan keluaran berupa nilai Boolean (True atau False).

2.6.4. Operator Logika

Operator logika digunakan untuk menggabungkan dua nilai kondisi atau lebih dan menghasilkan nilai boolean true atau false.

Operator	Keterangan
And	Menghasilkan true jika kedua kondisi bernilai true
Or	Menghasilkan true jika satu atau kedua kondisi bernilai true
Not	Membalikkan kondisi true menjadi false dan sebaliknya

Contoh penggunaan operator logika:

```
a = True
b = False

operator_and = a and b
operator_or = a or b
operator_not = not operator_or
print("operator and bernilai", operator_and)
print("operator or bernilai", operator_or)
print("operator not bernilai", operator_not)
```

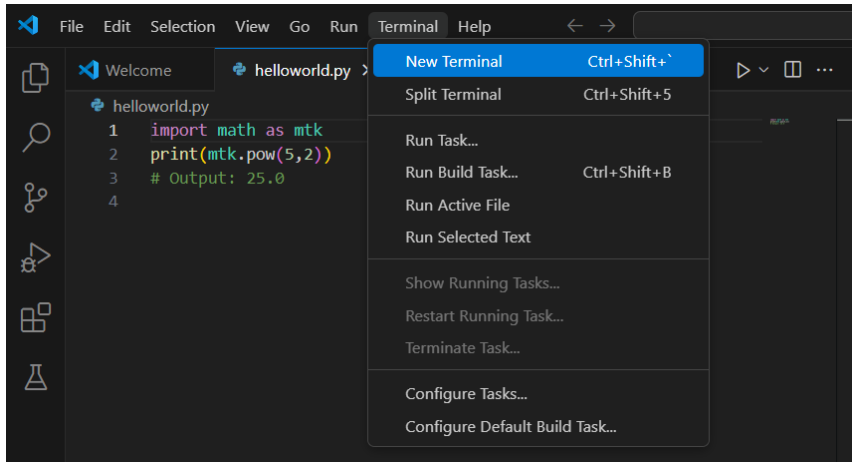
Program di atas merupakan contoh penggunaan operator logika di Python, yaitu `and`, `or`, dan `not`. Dua variabel Boolean didefinisikan: `a` bernilai `True` dan `b` bernilai `False`. Kemudian dilakukan tiga operasi logika:

1. `operator_and = a and b` → hasilnya `False`, karena `operator and` hanya akan menghasilkan `True` jika kedua operand bernilai `True`.
2. `operator_or = a or b` → hasilnya `True`, karena `operator or` akan menghasilkan `True` jika salah satu operand bernilai `True`.
3. `operator_not = not operator_or` → hasilnya `False`, karena `not` akan membalik nilai boolean dari operand-nya, dalam hal ini `not True` menjadi `False`.

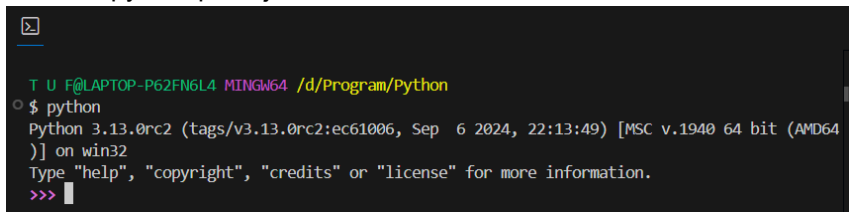
Terakhir, hasil dari ketiga operasi tersebut ditampilkan ke layar. Program ini menunjukkan bagaimana operator logika bekerja untuk memproses nilai boolean dalam pengambilan keputusan atau kondisi logis.

2.7 Penggunaan Interpreter

1. Masuk ke Visual Studio Code
2. Pilih menu terminal kemudian klik new terminal



3. Ketik python pada jendela terminal



4. Kemudian tulis kode program, disini membuat variabel angka1 bernilai 10 yang kemudian penambahan nilai sejumlah 2 dengan menggunakan dan mencetak nilai variabel angka1.

```
T U F@LAPTOP-P62FN6L4 MINGW64 /d/Program/Python
$ python
Python 3.13.0rc2 (tags/v3.13.0rc2:ec61006, Sep  6 2024, 22:13:49) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> angka1=10
>>> angka1+=2
>>> print(angka1)
12
```

2.8 Latihan

1. Buatlah variabel untuk menyimpan:

- Nama lengkap
- Umur
- Tinggi badan
- Status mahasiswa (True/False)

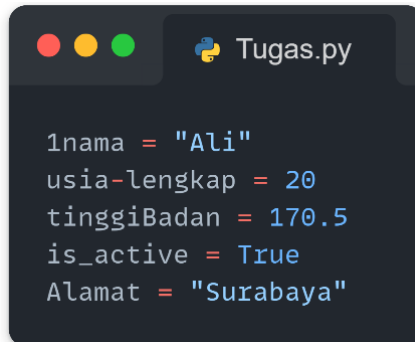
Tampilkan semua variabel tersebut menggunakan print().

```
Tugas.py

# Mendefinisikan variabel
nama_lengkap = "Budi Santoso"
umur = 20
tinggi_badan = 170.5
status_mahasiswa = True

# Menampilkan isi variabel
print("Nama Lengkap:", nama_lengkap)
print("Umur:", umur, "tahun")
print("Tinggi Badan:", tinggi_badan, "cm")
print("Status Mahasiswa:", status_mahasiswa)
```

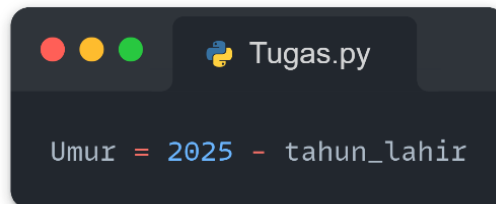
2. Tentukan apakah penulisan nama variabel berikut **benar** atau **salah** menurut aturan Python. Jika salah, jelaskan alasannya:



```
1nama = "Ali"  
usia-lengkap = 20  
tinggiBadan = 170.5  
is_active = True  
Alamat = "Surabaya"
```

3. Buat program yang meminta input dari pengguna:
- Nama
 - Tahun lahir

Hitung dan tampilkan umur pengguna berdasarkan tahun sekarang (misal 2025):



```
Umur = 2025 - tahun_lahir
```

4. Buatlah program yang menerima dua bilangan bulat dari pengguna, lalu hitung:
- Penjumlahan
 - Pengurangan
 - Perkalian
 - Pembagian
 - Sisa bagi (modulus)
 - Pemangkatan

```
Tugas.py

# Input dua bilangan dari pengguna
bil1 = int(input("Masukkan bilangan pertama: "))
bil2 = int(input("Masukkan bilangan kedua: "))

# Operasi aritmatika
penjumlahan = bil1 + bil2
pengurangan = bil1 - bil2
perkalian = bil1 * bil2
pembagian = bil1 / bil2
modulus = bil1 % bil2
pangkat = bil1 ** bil2

# Menampilkan hasil
print("Hasil Penjumlahan:", penjumlahan)
print("Hasil Pengurangan:", pengurangan)
print("Hasil Perkalian:", perkalian)
print("Hasil Pembagian:", pembagian)
print("Sisa Bagi (Modulus):", modulus)
print("Hasil Pemangkatan:", pangkat)
```

5. Tuliskan hasil akhir dari potongan kode berikut:

```
Tugas.py

nilai = 10
nilai += 5
nilai *= 2
nilai -= 4
print(nilai)
```

2.9 Tugas

1. Membuat program sederhana operasi aritmatika dengan dua input bilangan dari user. Hitung hasil dari:
 - a. Penjumlahan dari dua bilangan tersebut
 - b. Pengurangan dari dua bilangan tersebut
 - c. Perkalian dari dua bilangan tersebut
 - d. Pembagian dari dua bilangan tersebut

```
T U F@LAPTOP-P62FN6L4 MINGW64 /d/Program/Python
$ "C:/Users/T U F/AppData/Local/Programs/Python/Python313/python.exe" d:/Program/Python/aritmatika.py
Masukkan nilai untuk bilangan pertama: 2
Masukkan nilai untuk bilangan kedua: 3
Hasil Penjumlahan: 5
Hasil Pengurangan: -1
Hasil Perkalian: 6
Hasil Pembagian: 0.6666666666666666
```

2. Buatlah suatu program untuk menghitung luas dan keliling lingkaran dari input user berupa jari-jari dan gunakan fungsi math.

```
T U F@LAPTOP-P62FN6L4 MINGW64 /d/Program/Python
$ "C:/Users/T U F/AppData/Local/Programs/Python/Python313/python.exe" "d:/Program/Python/luas dan keliling lingkaran.py"
Masukkan jari-jari:
7
Luas Lingkaran : 153.93804002589985
Keliling Lingkaran : 43.982297150257104
```

3. Buatlah suatu program untuk menghitung luas permukaan dan volume kubus dari input user berupa jari-jari dan gunakan fungsi math.

```
T U F@LAPTOP-P62FN6L4 MING64 /d/Program/Python
$ "C:/Users/T U F/AppData/Local/Programs/Python/Python313/python.exe" "d:/Program/Python/luas
s kubus.py"
Masukkan jari-jari:
3
Luas Permukaan Kubus : 54.0
Volume Kubus : 27.0
```

4. Buatlah program sederhana yang:
- Meminta input panjang dan lebar dari pengguna.
 - Menghitung luas dan keliling persegi panjang.
 - Menampilkan hasilnya ke layar.

Rumus:

- Luas = panjang $\times$ lebar
- Keliling = $2 \times (\text{panjang} + \text{lebar})$

BAB III

CONDITIONAL DAN LOGICAL OPERATOR

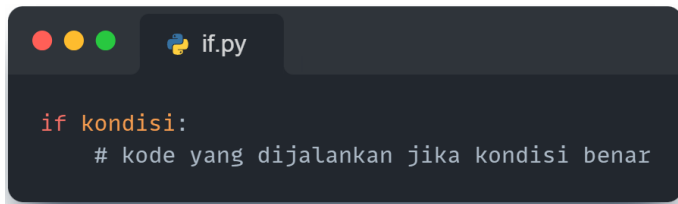
Tujuan Pembelajaran :

1. Memahami konsep dasar percabangan menggunakan if, elif, dan else.
2. Menggunakan operator logika seperti and, or, dan not.
3. Mampu mengimplementasikan percabangan dan operator logika dalam program sederhana.

3.1 Struktur Dasar Percabangan

1. If Statement

Pernyataan if digunakan untuk mengevaluasi suatu kondisi. Jika kondisi tersebut bernilai True, maka blok kode di dalam pernyataan if akan dieksekusi. Jika kondisi bernilai False, program akan melanjutkan ke pernyataan berikutnya. Contoh:



```
if kondisi:
    # kode yang dijalankan jika kondisi benar
```

2. If-Else Statement

Pernyataan if-else memberikan alternatif. Jika kondisi dalam if bernilai True, maka blok kode di dalam if akan dieksekusi. Namun, jika kondisi tersebut bernilai False, maka blok kode di dalam else akan dijalankan. Contoh:



```
if kondisi:
    # kode jika kondisi benar
else:
    # kode jika kondisi salah
```

3. If-Elif-Else Statement

Pernyataan elif (singkatan dari "else if") digunakan ketika ada beberapa kondisi yang perlu dievaluasi. Program akan memeriksa setiap kondisi secara berurutan. Jika salah satu kondisi bernilai True, blok kode yang sesuai akan dieksekusi, dan program akan melewati sisa kondisi. Contoh:



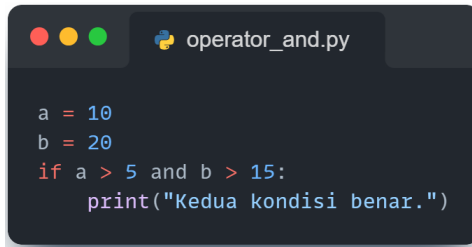
```
if kondisi1:
    # kode jika kondisi1 benar
elif kondisi2:
    # kode jika kondisi2 benar
else:
    # kode jika semua kondisi salah
```

3.2 Operator Logika

Operator logika dalam pemrograman Python digunakan untuk melakukan operasi pada nilai boolean (True atau False). Operator ini sangat berguna dalam percabangan, di mana kita perlu mengevaluasi beberapa kondisi sekaligus. Ada tiga operator logika utama yang sering digunakan dalam Python: and, or, dan not. Mari kita bahas masing-masing operator ini secara mendetail.

1. AND

Operator and mengembalikan True hanya jika kedua kondisi yang dievaluasi bernilai True. Jika salah satu atau kedua kondisi bernilai False, maka hasilnya adalah False. Contoh



```
a = 10
b = 20
if a > 5 and b > 15:
    print("Kedua kondisi benar.")
```

Dalam contoh di atas, karena kedua kondisi ($a > 5$ dan $b > 15$) bernilai True, maka output yang dihasilkan adalah "Kedua kondisi benar."

2. OR

Operator or mengembalikan True jika salah satu dari kondisi yang dievaluasi bernilai True. Hanya jika kedua kondisi bernilai False, maka hasilnya adalah False. Contoh:

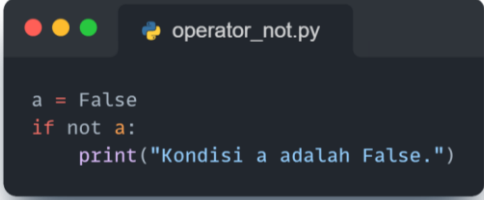


```
a = 10
b = 5
if a > 15 or b < 10:
    print("Salah satu kondisi benar.")
```

Dalam contoh ini, meskipun $a > 15$ adalah False, $b < 10$ adalah True, sehingga output yang dihasilkan adalah "Salah satu kondisi benar."

3. NOT

Operator not digunakan untuk membalikkan nilai boolean. Jika kondisi bernilai True, maka not akan mengembalikan False, dan sebaliknya. Contoh:



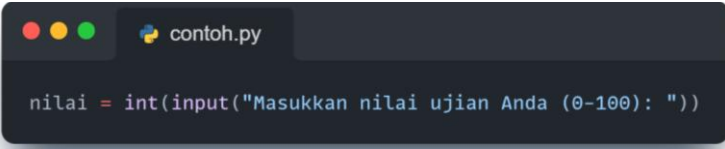
```
a = False
if not a:
    print("Kondisi a adalah False.")
```

Dalam contoh ini, karena a bernilai False, maka not a akan bernilai True, sehingga output yang dihasilkan adalah "Kondisi a adalah False."

3.1 Latihan

Berikut adalah contoh program dalam Python yang meminta pengguna untuk memasukkan nilai ujian dan status kehadiran, kemudian memberikan kategori berdasarkan kriteria yang telah ditentukan.

1. Program meminta pengguna untuk memasukkan nilai ujian dalam rentang 0 hingga 100.



```
nilai = int(input("Masukkan nilai ujian Anda (0-100): "))
```

2. Program meminta pengguna untuk memasukkan status kehadiran, apakah "hadir" atau "tidak hadir".

```
contoh.py

nilai = int(input("Masukkan nilai ujian Anda (0-100): "))

kehadiran = input("Apakah Anda hadir dalam ujian? (hadir/tidak hadir): ")
```

3. Percabangan:

- Jika nilai ≥ 75 dan status kehadiran adalah "hadir", maka program mencetak "Kategori: Lulus".
- Jika nilai < 75 dan status kehadiran adalah "hadir", maka program mencetak "Kategori: Remedial".
- Jika status kehadiran adalah "tidak hadir", maka program mencetak "Kategori: Tidak Lulus".

```
contoh.py

nilai = int(input("Masukkan nilai ujian Anda (0-100): "))

kehadiran = input("Apakah Anda hadir dalam ujian? (hadir/tidak hadir): ")

if nilai  $\geq$  75 and kehadiran.lower() == "hadir":
    print("Kategori: Lulus")
elif nilai < 75 and kehadiran.lower() == "hadir":
    print("Kategori: Remedial")
else:
    print("Kategori: Tidak Lulus")
```

3.2 Tugas Praktikum

1. Buatlah program yang meminta pengguna memasukkan angka dan menentukan apakah angka tersebut genap atau ganjil.

Contoh output :

```
Masukkan sebuah angka: 13
Angka 13 adalah Ganjil.
```

2. Kembangkan program yang meminta pengguna memasukkan nilai ujian (0-100) dan memberikan feedback sebagai berikut:

Nilai 90-100: "Sangat Baik"

Nilai 80-89: "Baik"

Nilai 70-79: "Cukup"

Nilai 60-69: "Kurang"

Nilai di bawah 60: "Sangat Kurang"

Contoh output :

```
Masukkan nilai ujian Anda (0-100): 80
Feedback: Baik
```

3. Buatlah program yang meminta pengguna memasukkan usia dan tekanan darah. Tentukan status kesehatan sebagai berikut:

Jika usia ≥ 60 dan tekanan darah > 140 , maka "Tinggi".

Jika usia ≥ 60 dan tekanan darah ≤ 140 , maka "Normal".

Jika usia < 60 dan tekanan darah > 130 , maka "Tinggi".

Jika usia < 60 dan tekanan darah ≤ 130 , maka "Normal".

Contoh output

```
Masukkan usia Anda: 40  
Masukkan tekanan darah Anda: 120  
Status Kesehatan: Normal
```

4. Buatlah program yang meminta pengguna memasukkan tiga bilangan, lalu menentukan bilangan terbesar di antara ketiganya.

Petunjuk:

Gunakan operator perbandingan ($>$) dan struktur percabangan (if, elif, else) untuk menentukan mana dari ketiga bilangan yang paling besar.

BAB IV LOOPING

Tujuan Pembelajaran :

1. Mahasiswa memahami proses perulangan menggunakan pernyataan for, while dan fungsi range
2. Mahasiswa mampu menggunakan operator perulangan
3. Mahasiswa mampu mengimplementasikan perulangan dalam program sederhana

4.1 Struktur Dasar Perulangan

1. Perulangan while

Perulangan while digunakan ketika pengulangan harus dilakukan selama suatu kondisi bernilai True. Loop akan terus berjalan selama kondisi yang ditentukan tetap benar, dan berhenti ketika kondisi tersebut menjadi False.

Contoh:



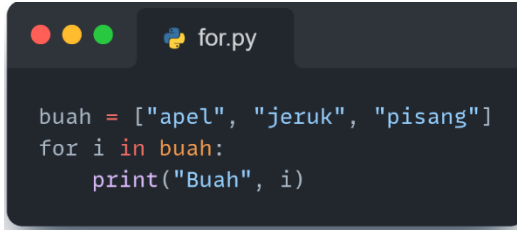
```
iterasi = 1
while iterasi < 5:
    print("Ini adalah nomor: ", iterasi)
    iterasi+=1
print("Selesai!!!")
```

Program ini akan menjalankan print("Ini adalah nomor: ", iterasi) selama iterasi masih lebih kecil dari 5, iterasi ditambah 1 setiap kondisi True. Pada saat nilai iterasi mencapai 5, maka kondisi False, dan program akan keluar dari looping while dan melanjutkan baris selanjutnya yaitu print("Selesai!!!")

2. Perulangan for

Perulangan for digunakan untuk mengulangi suatu blok kode berupa array atau himpunan.

Contoh:



```
buah = ["apel", "jeruk", "pisang"]
for i in buah:
    print("Buah", i)
```

Dalam contoh diatas akan menjalankan `print("Buah", i)` selama array atau himpunan di dalam variabel buah selesai dijalankan semua. Ketika array sudah selesai dijalankan semua maka program akan keluar dari looping for

3. Fungsi range()

Fungsi `range()` dapat digunakan untuk menghasilkan deret bilangan. `range(10)` akan menghasilkan bilangan dari 0 sampai dengan 9 (10 bilangan). Kita juga bisa menentukan batas bawah, batas atas, dan interval dengan format `range(batas bawah, batas atas, interval)`. Bila interval dikosongkan, maka nilai default 1 yang akan digunakan.

Contoh:



```
for i in range(6):
    print(i)
```

Program tersebut menggunakan perulangan `for` dengan fungsi `range(6)`, yang berarti akan menghasilkan angka dari 0 hingga 5. Pada setiap iterasi, nilai dari variabel `i` akan bertambah satu secara otomatis, dimulai dari 0. Fungsi `print(i)` digunakan untuk mencetak nilai `i` ke layar pada setiap putaran perulangan. Karena `range(6)` menghasilkan enam angka (0, 1, 2, 3, 4, dan 5), maka program ini akan mencetak enam baris angka.



```
for i in range(0,11,2)
    print(i)
```

Program tersebut menggunakan perulangan `for` bersama fungsi `range(0, 11, 2)`, yang berarti akan menghasilkan urutan angka dari 0 sampai kurang dari 11, dengan kenaikan (step) 2 pada setiap iterasi. Nilai-nilai yang dihasilkan oleh `range(0, 11, 2)` adalah: 0, 2, 4, 6, 8, dan 10. Pada setiap putaran, nilai `i` akan dicetak ke layar menggunakan fungsi `print(i)`. Dengan demikian, program ini akan mencetak angka genap dari 0 hingga 10, masing-masing pada baris yang berbeda. Pola perulangan ini sangat berguna ketika ingin mencetak deret angka tertentu secara bertahap, misalnya angka genap, angka kelipatan, atau langkah terkontrol lainnya.

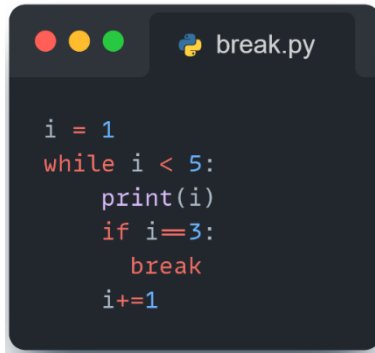
4.2 Kendali Perulangan

Perulangan umumnya akan berhenti bila kondisi sudah bernilai `False`. Akan tetapi, seringkali kita perlu keluar dari looping di tengah jalan tergantung keperluan. Hal ini bisa kita lakukan dengan menggunakan statement **`break`** dan **`continue`**.

1. Statement break

Pada program perulangan for dan while ada yang disebut sebagai statement **Break** yang merupakan pernyataan yang akan membuat sebuah program berhenti atau keluar dari perulangan.

Contoh:



```
i = 1
while i < 5:
    print(i)
    if i==3:
        break
    i+=1
```

Dalam contoh ini, looping akan dijalankan dan ketika nilai dari i sama dengan 3 maka looping akan berhenti atau keluar dari perulangan.

2. Statement continue

Selain statement break, dalam python juga ada statement **continue** yang bertugas jika ada program yang sama dengan statement tidak akan di eksekusi. Atau bisa kita katakan untuk melewati perintah yang ada. Statement continue menyebabkan program langsung melanjutkan ke step/interval berikutnya dan mengabaikan (skip) baris kode dibawahnya (yang satu blok).

Contoh:

```
continue.py

for i in range(1,6):
    if i == 3:
        continue
    print(i)
```

Dalam contoh ini, looping akan dijalankan dan ketika nilai dari *i* sama dengan 3 maka program langsung melanjutkan ke step berikutnya

4.3 Latihan

1. Berikut adalah contoh program menggunakan perulangan **for** yang meminta pengguna untuk menginputkan data perulangan sesuai dengan kebutuhannya

```
contoh.py

masukkan = int(input("Masukkan jumlah data yang ingin diinputkan: "))
for i in range(masukkan):
    print ("data Ke - " + str(i+1))
    nama=input("Masukkan Nim anda : ")
    uts=int(input("Masukkan Nilai UTS anda : "))
    uas=int(input("Masukkan Nilai UAS : "))
    print("NIM anda adalah",nama,"nilai UTS anda",uts,"nilai UAS anda",uas)
    print("-----\n")
```

2. Berikut contoh program menggunakan perulangan **while** untuk menentukan bilangan ganjil

```
contoh1.py

print("Program untuk menentukan bilangan ganjil")
batas = int(input("Masukkan batas angka: "))

i = 1
while i <= batas:
    if i % 2:
        print(i)
    i += 1
```

4.4 Tugas Praktikum

1. Buatlah program menggunakan **while** yang meminta **pengguna** memasukkan bilangan awal dan bilangan akhir untuk menentukan bilangan ganjil

```
Program untuk menentukan bilangan ganjil
Masukkan bilangan awal: 5
Masukkan bilangan akhir: 10
5
7
9
```

2. Buatlah program menggunakan **while** yang meminta pengguna memasukkan sebuah angka, lalu mencetak angka tersebut berulang kali sampai jumlah total cetakan mencapai angka tersebut.

```
Masukkan sebuah angka: 4
4
4
4
4
```

3. Buatlah program menggunakan **for** yang meminta pengguna untuk memasukkan banyaknya mahasiswa dan banyak mata kuliah yang diambil, kemudian masukkan nilai setiap mata kuliah dan hitung rata-rata nilainya

Rumus rata-rata = total nilai/banyak mata kuliah

```
Banyak Mahasiswa : 2
Mahasiswa ke 1
Banyak mata kuliah yang diambil: 2
Input nilai Matkul ke 1: 75
Input nilai Matkul ke 2: 80
Rata-rata: 77.5
-----

Mahasiswa ke 2
Banyak mata kuliah yang diambil: 2
Input nilai Matkul ke 1: 60
Input nilai Matkul ke 2: 90
Rata-rata: 75.0
-----
```


BAB V FUNGSI

Tujuan Pembelajaran :

- 1. Mahasiswa dapat memahami perbedaan antara fungsi tanpa nilai balik (void) dan fungsi dengan nilai balik untuk mengatur alur data dalam program.**
- 2. Mahasiswa mampu membuat fungsi dengan default argument untuk memberikan fleksibilitas saat pemanggilan fungsi.**
- 3. Mahasiswa dapat menggunakan variable length argument untuk menangani jumlah parameter yang tidak tetap dalam fungsi.**
- 4. Mahasiswa memahami konsep pass by object reference untuk mengetahui bagaimana data diubah saat melewati fungsi.**
- 5. Mahasiswa memahami variable scope, baik lokal maupun global, dan dampaknya terhadap pengelolaan variabel dalam program.**

5.1 Pengenalan Fungsi

Fungsi dalam Python adalah blok kode yang dirancang untuk melakukan tugas tertentu dan dapat digunakan kembali. Jika blok kode yang sama digunakan berulang kali, fungsi memungkinkan programmer untuk menulis blok kode tersebut sekali, memberinya nama, dan menggunakan kode tersebut sebanyak yang diperlukan dengan memanggil blok tersebut berdasarkan namanya. Fungsi dapat menerima nilai sebagai

masukan dan mengembalikan nilai untuk menjalankan tugas, termasuk perhitungan yang kompleks. Secara umum, struktur fungsi terdiri dari:

- Nama Fungsi
- Parameter (opsional): Data yang diterima fungsi untuk diproses.
- Statemen/Isi Fungsi: Instruksi yang akan dijalankan.
- Nilai Kembali (opsional): Hasil yang dikembalikan oleh fungsi.

```
def nama_fungsi(parameter):  
    statement  
    nilai kembali
```

5.2 Penggunaan Fungsi

1. Fungsi Tanpa Nilai Balik

Fungsi tanpa nilai balik adalah fungsi yang hanya menjalankan perintah tetapi tidak mengembalikan nilai apa pun ke pemanggilnya. Fungsi ini biasanya digunakan untuk mencetak hasil atau memodifikasi data global.

```
def pesan():  
    print("Halo Dunia!")  
  
pesan()
```

Dalam program diatas menggunakan fungsi pesan yang akan menampilkan output ("Halo Dunia!")

2. Fungsi dengan Nilai Balik

Fungsi dengan nilai balik menggunakan perintah `return` untuk mengembalikan hasil ke pemanggilnya. Fungsi ini cocok untuk melakukan perhitungan yang hasilnya dibutuhkan di tempat lain.

```
def perkalian(a, b):  
    return a * b  
  
hasil = perkalian(5, 3)  
print("Hasil perkalian:", hasil)
```

Program diatas merupakan Fungsi `perkalian(a, b)` didefinisikan untuk menerima dua parameter, yaitu `a` dan `b`, lalu mengembalikan hasil perkalian keduanya menggunakan `return a * b`. Di bagian bawah program, fungsi tersebut dipanggil dengan argumen 5 dan 3, sehingga `perkalian(5, 3)` menghasilkan nilai 15. Nilai ini disimpan dalam variabel `hasil`. Selanjutnya, fungsi `print()` digunakan untuk menampilkan output ke layar dengan format: "Hasil perkalian: 15".

3. Default Argument

Default argument memberikan nilai bawaan pada parameter. Jika tidak ada nilai yang diberikan saat fungsi dipanggil, nilai default akan digunakan.

```
def sapa(nama, pesan="Selamat pagi"):  
    print(f"{pesan}, {nama}!")  
  
sapa("Budi")  
sapa("Siti", "Selamat siang")
```

Program di atas merupakan contoh penggunaan fungsi dengan parameter default di Python. Fungsi bernama `sapa()` memiliki dua parameter: `nama` dan `pesan`, di mana parameter `pesan` memiliki nilai default "Selamat pagi". Fungsi ini mencetak kalimat sapaan dengan format "{pesan}, {nama}!" menggunakan f-string untuk menyisipkan nilai variabel ke dalam string.

Saat fungsi `sapa("Budi")` dipanggil, hanya argumen `nama` yang diberikan, sehingga Python menggunakan nilai default "Selamat pagi" untuk parameter `pesan`, dan hasilnya adalah:

"Selamat pagi, Budi!"

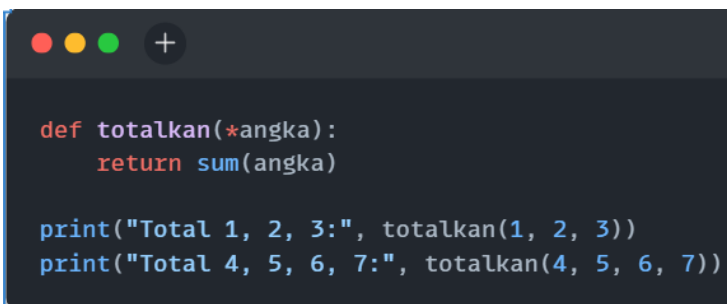
Sementara pada pemanggilan kedua `sapa("Siti", "Selamat siang")`, kedua parameter diisi, sehingga output-nya menjadi:

"Selamat siang, Siti!"

4. Length Argument pada Fungsi

a. Variable Length Argument

Python memungkinkan fungsi menerima se jumlah argumen yang tidak terbatas tanpa kata kunci menggunakan perintah `*args`.



```
def totalkan(*angka):  
    return sum(angka)  
  
print("Total 1, 2, 3:", totalkan(1, 2, 3))  
print("Total 4, 5, 6, 7:", totalkan(4, 5, 6, 7))
```

Fungsi `totalkan(*angka)` memungkinkan pengguna untuk memasukkan jumlah argumen sebanyak apa pun. Di dalam fungsi, digunakan fungsi bawaan `sum()` untuk menghitung total dari semua nilai yang diterima. Pada baris pertama pemanggilan, `totalkan(1, 2, 3)` akan mengembalikan jumlah 6, sedangkan pada baris kedua `totalkan(4, 5, 6, 7)` akan menghasilkan 22.

b. Keyword Variable Length Argument

Keyword variable-length arguments dalam Python mengacu pada kemampuan fungsi untuk menerima jumlah argumen kata kunci yang tidak terbatas menggunakan sintaks **`**kwargs`**. Argumen ini dikumpulkan dalam bentuk dictionary di dalam fungsi, memungkinkan pemetaan pasangan kunci-nilai.

```
def profil(**info):  
    for key, value in info.items():  
        print(f"{key}: {value}")  
  
profil(nama="Budi", usia=25, pekerjaan="Programmer")
```

Program diatas merupakan contoh penggunaan parameter tak terbatas berupa pasangan kunci dan nilai (keyword arguments) menggunakan **`**kwargs`** dalam Python. Fungsi `profil(**info)` dapat menerima sejumlah argumen berupa pasangan `key=value` tanpa jumlah yang ditentukan. Semua argumen tersebut disimpan dalam bentuk dictionary dengan nama `info`. Di dalam fungsi, dilakukan perulangan terhadap `info.items()` untuk mengambil setiap pasangan `key` dan `value`, lalu mencetaknya dengan format `key: value`. Pada pemanggilan fungsi `profil (nama="Budi", usia=25, pekerjaan="Programmer")`, fungsi akan mencetak tiga baris:

nama: Budi
usia: 25
pekerjaan: Programmer

5. Keyword Argument

Keyword arguments memungkinkan pemanggilan fungsi dengan menyebutkan nama parameter secara eksplisit sehingga urutan argumen tidak masalah.

```
def biodata(nama, usia):  
    print(f>Nama: {nama}, Usia: {usia}")  
  
biodata(usia=19, nama="Ali")
```

Program di atas merupakan contoh penggunaan fungsi dengan parameter bernama (keyword arguments) dalam Python. Fungsi `biodata(nama, usia)` memiliki dua parameter: `nama` dan `usia`, lalu mencetak keduanya menggunakan format string `f>Nama: {nama}, Usia: {usia}"`. Yang menarik, saat memanggil fungsi `biodata`, argumen diberikan dengan cara tidak berurutan (`usia=19, nama="Ali"`), namun hasilnya tetap benar karena Python menggunakan nama parameter, bukan urutannya.

Ketika program dijalankan, akan ditampilkan output:

Nama: Ali, Usia: 19

6. Pass By Object Reference

Python menggunakan mekanisme `pass by object reference`. Untuk tipe data mutable seperti `list`, perubahan akan memengaruhi data asli. Sedangkan untuk tipe data immutable seperti `integer`, hanya salinan nilai yang diubah.

```
def ubah_list(data):  
    data.append(100)  
  
def ubah_integer(angka):  
    angka += 100  
    return angka  
  
list_angka = [1, 2, 3]  
angka = 10  
  
ubah_list(list_angka)  
print("List setelah fungsi:", list_angka)  
  
hasil_angka = ubah_integer(angka)  
print("Angka setelah fungsi:", angka)  
print("Hasil fungsi:", hasil_angka)
```

Program di atas menunjukkan perbedaan perilaku parameter bertipe list dan integer saat digunakan dalam fungsi Python. Fungsi `ubah_list(data)` menerima parameter berupa list, kemudian menambahkan nilai 100 ke dalam list tersebut menggunakan `append()`. Karena list adalah tipe data mutable (bisa berubah), perubahan yang terjadi di dalam fungsi akan memengaruhi variabel `list_angka` di luar fungsi. Maka, setelah fungsi `ubah_list()` dipanggil, `list_angka` yang awalnya `[1, 2, 3]` menjadi `[1, 2, 3, 100]`.

Sementara itu, fungsi `ubah_integer(angka)` menerima parameter berupa integer dan menambahkan 100 ke dalamnya. Namun, karena integer adalah tipe data immutable (tidak bisa diubah langsung), nilai asli dari variabel `angka` tidak berubah. Perubahan hanya terjadi pada variabel lokal di dalam fungsi, dan hasil akhirnya dikembalikan melalui `return`.

Oleh karena itu, saat dijalankan, program akan mencetak:

List setelah fungsi: [1, 2, 3, 100]

Angka setelah fungsi: 10

Hasil fungsi: 110

7. Ruang Lingkup Variabel (Variable Scope)

Ruang lingkup variabel menentukan lokasi di mana suatu variabel dapat digunakan. Variabel global dapat diakses di seluruh bagian program, sedangkan variabel lokal hanya tersedia di dalam fungsi tempat variabel tersebut dideklarasikan.

```
x = 10 # variabel Global

def scope_demo():
    x = 5 # variabel Lokal
    print("Variabel lokal x:", x)

scope_demo()
print("Variabel global x:", x)
```

5.3. Latihan

1. Buatlah sebuah fungsi bernama `sapa()` yang mencetak kalimat:
"Halo, selamat datang di dunia Python!"
Panggil fungsi tersebut dari program utama.

```
Tugas.py

def sapa():
    print("Halo, selamat datang di dunia Python!")

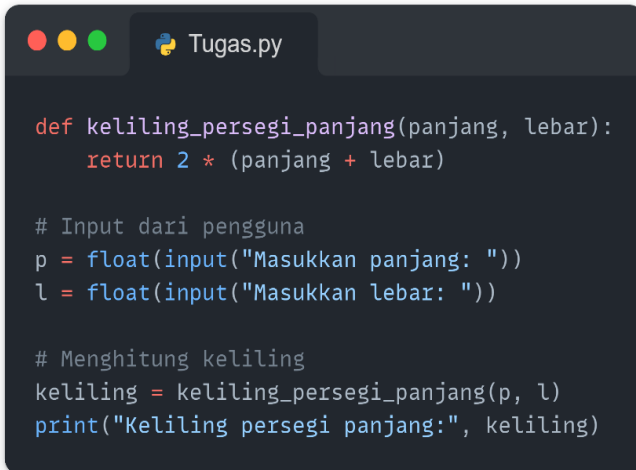
# Memanggil fungsi
sapa()
```

2. Buatlah fungsi `luas_persegi(sisi)` yang menerima satu parameter, dan mengembalikan nilai luas dari persegi.
Tampilkan hasil pemanggilan fungsi dengan `sisi = 8`.



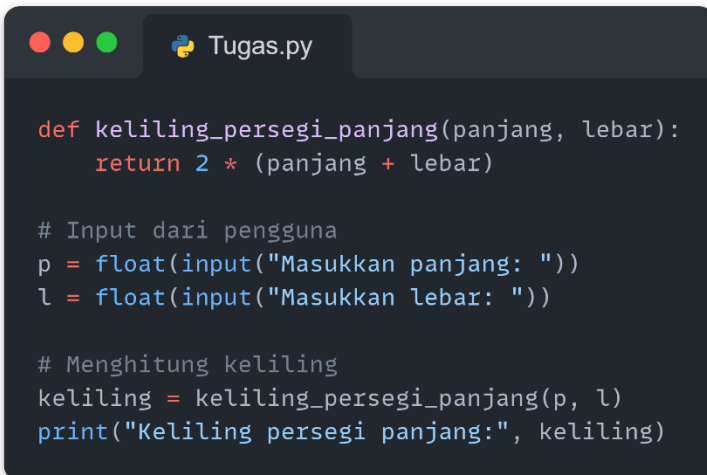
```
def luas_persegi(sisi):  
    return sisi * sisi  
  
# Memanggil fungsi  
hasil = luas_persegi(8)  
print("Luas persegi:", hasil)
```

3. Buat fungsi `keliling_persegi_panjang(panjang, lebar)` yang menghitung keliling berdasarkan rumus:
 $\text{keliling} = 2 \times (\text{panjang} + \text{lebar})$
Inputkan panjang dan lebar dari pengguna, lalu tampilkan hasilnya.



```
def keliling_persegi_panjang(panjang, lebar):  
    return 2 * (panjang + lebar)  
  
# Input dari pengguna  
p = float(input("Masukkan panjang: "))  
l = float(input("Masukkan lebar: "))  
  
# Menghitung keliling  
keliling = keliling_persegi_panjang(p, l)  
print("Keliling persegi panjang:", keliling)
```


4. Buat program sederhana dengan struktur fungsi:
- Fungsi `input_data()` → menerima nama dan nilai
 - Fungsi `kategori_nilai(nilai)` → menentukan kategori:
 - ≥ 90 : Sangat Baik
 - ≥ 75 : Baik
 - ≥ 60 : Cukup
 - < 60 : Kurang
 - Fungsi `tampilkan(nama, nilai, kategori)` → mencetak hasil akhir



```
def keliling_persegi_panjang(panjang, lebar):  
    return 2 * (panjang + lebar)  
  
# Input dari pengguna  
p = float(input("Masukkan panjang: "))  
l = float(input("Masukkan lebar: "))  
  
# Menghitung keliling  
keliling = keliling_persegi_panjang(p, l)  
print("Keliling persegi panjang:", keliling)
```

5.4. Tugas Praktikum

1. Tulis sebuah fungsi bernama `luas_persegi_panjang()` yang menerima dua parameter: panjang dan lebar dari input user menggunakan python. Fungsi ini mengembalikan luas persegi panjang.

```
Masukkan panjang: 20
Masukkan lebar: 10
Luas persegi panjang adalah: 200.0
```

2. Anda bekerja pada sistem manajemen inventaris untuk sebuah toko online. Tugas Anda adalah menulis sebuah program yang mengelola pengurangan stok barang setiap kali ada pembelian. Program ini harus memungkinkan pengguna untuk mengurangi jumlah stok barang yang tersedia dengan memastikan bahwa jumlah stok yang ingin dikurangi tidak melebihi stok yang ada.
 - Buat variabel global `stok_barang` berjumlah 100 untuk menyimpan jumlah stok barang awal.
 - Tulis fungsi `kurangi_stok()` yang mengurangi stok barang berdasarkan input jumlah dari pengguna dengan memastikan stok mencukupi.
 - Program utama harus meminta input dari pengguna mengenai jumlah barang yang ingin dikurangi dan memproses transaksi tersebut.
 - Pastikan bahwa jika stok tidak cukup, transaksi pembelian tidak dapat dilakukan.

```
• Sistem Manajemen Stok Barang Toko
Masukkan jumlah barang yang ingin dikurangi: 20
20 barang telah terjual. Stok total: 80
Apakah Anda ingin melanjutkan transaksi? (ya/tidak): ya
Masukkan jumlah barang yang ingin dikurangi: 10
10 barang telah terjual. Stok total: 70
Apakah Anda ingin melanjutkan transaksi? (ya/tidak): tidak
Transaksi selesai. Terima kasih!
```

3. Buatlah sebuah fungsi bernama `konversi_suhu()` yang meminta input suhu dalam derajat Celcius dari pengguna, lalu mengonversinya ke Fahrenheit dan Kelvin. Fungsi ini harus mengembalikan kedua hasil konversi.

Gunakan rumus:

Fahrenheit = $(9/5 \times \text{Celcius}) + 32$

Kelvin = Celcius + 273.15

BAB VI

FILE

Tujuan Pembelajaran :

1. Memahami konsep dasar file handling di Python.
2. Menggunakan Pandas untuk membaca file CSV, Excel, dan file teks.
3. Menulis dan menyimpan file data ke berbagai format menggunakan Pandas.
4. Melakukan manipulasi sederhana pada data menggunakan Pandas.

6.1 File Handling

A. Konsep File Handling di Python

File handling adalah kemampuan Python untuk bekerja dengan file seperti membaca, menulis, dan memperbarui data yang tersimpan. File dapat berupa teks, CSV, Excel, JSON, atau format lainnya. Fungsi awal untuk dapat melakukan operasi file adalah `open()` yang mempunyai 2 parameter yaitu `filename` dan `mode`. `filename` adalah nama file yang ingin diproses sedangkan `mode` adalah metode operasi yang ingin dilakukan. Ada 3 jenis metode operasi dasar di Python, antara lain :

Mode	Keterangan
r	Membuka file untuk membaca (default).
w	Membuka file untuk menulis (menghapus isinya).
a	Membuka file untuk menambahkan data.

1. Membaca File

Python menyediakan fungsi `open()` untuk membuka file. Fungsi ini memerlukan nama file dan mode yang diinginkan. Python menyediakan metode untuk membaca file, yaitu

- `read()` : Membaca seluruh isi file.
- `readline()` : Membaca satu baris.
- `readlines()` : Membaca semua baris sebagai list.

```
with open('events.txt', 'r') as file:
    content = file.read() # Membaca seluruh isi file
    print(content)
```

Pastikan file `events.txt` berada dalam 1 folder dengan kode di atas. Selanjutnya kita jalankan dan hasilnya adalah

```
Training,30/12/23
Spanish Test,20/12/23
School Trip,15/01/24
My Birthday,21/12/23
```

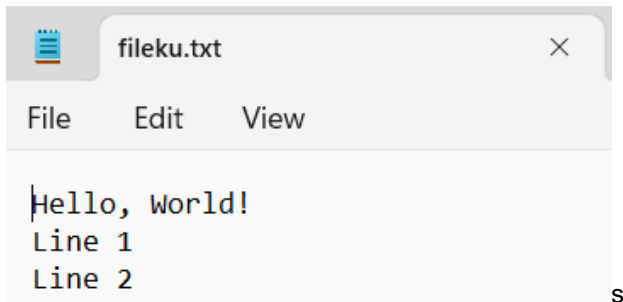
2. Menulis ke File

Operasi `write` memberikan kita akses untuk menulis ke dalam file baru. Metode untuk menulis data ke file adalah `write()` dan `writelines()`.

- `write()` : Menulis string ke file.
- `writelines()` : Menulis list string ke file.

```
with open('fileku.txt', 'w') as file:  
    file.write('Hello, World!\n')  
    file.writelines(['Line 1\n', 'Line 2\n'])
```

Setelah kode berhasil dijalankan maka akan terbuat file dengan nama fileku.txt dengan isi sebagai berikut

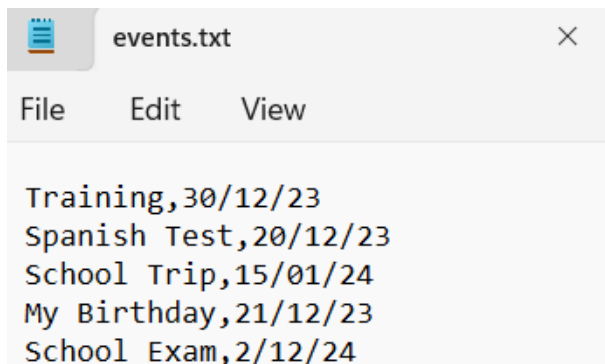


3. Menambahkan Data ke File

Mode 'a' digunakan untuk menambahkan data baru tanpa menghapus data lama.

```
with open('events.txt', 'a') as file:  
    file.write('\nSchool Exam,2/12/24')
```

Buka / reload kembali file events.txt maka isinya akan berubah seperti ini



```
Training,30/12/23
Spanish Test,20/12/23
School Trip,15/01/24
My Birthday,21/12/23
School Exam,2/12/24
```

B. Pengenalan Pandas

Pandas adalah paket libraries Python yang biasanya digunakan oleh praktisi data untuk mempermudah dalam mengolah dan menganalisis data-data terstruktur. Biasanya, data scientist, data engineer, hingga data analyst akan menggunakan libraries Pandas untuk memproses data, membersihkan data, manipulasi data, analisis data. Libraries Pandas ini dibangun atas dua libraries inti Python yaitu:

- Matplotlib = Untuk visualisasi data
- NumPy = Untuk operasi matematika

Libraries Pandas memiliki format data yang disebut DataFrame, jadi Pandas DataFrame adalah struktur data dua dimensi seperti tabel yang berisi baris dan kolom. Pandas DataFrame berfungsi untuk menyimpan data dalam format grid yang bisa diubah-ubah dengan fleksibilitas yang sangat besar. Gambaran sederhananya, Pandas DataFrame mirip dengan tabel yang ada di Microsoft Excel. Setiap baris dan kolom akan memiliki label yang bisa kamu gunakan untuk mengakses dan

memanipulasi data. Jadi, melalui Pandas DataFrame kamu bisa memanipulasi data, mengorganisir data, dan membersihkan data.

Langkah pertama bekerja menggunakan pandas adalah memastikan library tersebut sudah terpasang di folder Python. Jika belum terinstall, maka kita perlu menginstallnya di sistem kita menggunakan perintah pip.

```
pip install pandas
```

Setelah pandas diinstal ke dalam sistem, kita perlu mengimpor library ini agar bisa digunakan. Modul ini umumnya diimpor sebagai:

```
import pandas as pd
```

1. Membaca File CSV dengan Pandas

Gunakan fungsi `pd.read_csv()` untuk membaca file CSV ke dalam DataFrame.

```
import pandas as pd

# Membaca file CSV
df = pd.read_csv('data.csv')
print("Data:")
print(df)

# Menampilkan 5 baris pertama
print("\nBaris pertama:")
print(df.head())
```


Dengan output berikut

```
Data:
      nim      nama  nilai_alpro
0  22091397001  Asyifatul        90
1  22091397002   Roihan        95
2  22091397003    Insan        85
3  22091397004  Novrindah        85
4  22091397005   Furqon        88
5  22091397006    Hilal        83
6  22091397007   Ulung        88
7  22091397008   Yolanda        82
8  22091397009   Habibil        95
9  22091397010   Fajrul        93

Baris pertama:
      nim      nama  nilai_alpro
0  22091397001  Asyifatul        90
1  22091397002   Roihan        95
2  22091397003    Insan        85
3  22091397004  Novrindah        85
4  22091397005   Furqon        88
```

Gunakan parameter `sep` untuk mengganti delimiter default (,) dengan yang lain.

```
df.to_csv('output.txt', sep='|', index=False)
```

Separator adalah tanda pemisah yang digunakan untuk memisahkan elemen-elemen dalam file berbasis teks, seperti file CSV (Comma-Separated Values) atau file teks dengan format tabular lainnya. Separator membantu mengidentifikasi batas antar kolom atau elemen dalam data. Separator yang umum adalah (,), (\t), (|), (;)

Untuk mengetahui tipe data dari setiap kolom, gunakan fungsi `info()`

```
df.info()
```

dengan output berikut

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   nim          10 non-null    int64
1   nama         10 non-null    object
2   nilai_alpro  10 non-null    int64
dtypes: int64(2), object(1)
memory usage: 372.0+ bytes
```

Kita dapat menambahkan kolom baru dengan memberikan nama kolom dan datanya. Misalkan kita menambahkan kolom `nilai_uas` dengan nilai $0,3 \times \text{nilai_alpro}$

```
df['nilai_uas'] = df['nilai_alpro'] * 0.4
```

dengan output berikut

	nim	nama	nilai_alpro	nilai_uas
0	22091397001	Asyifatul	90	36.0
1	22091397002	Roihan	95	38.0
2	22091397003	Insan	85	34.0
3	22091397004	Novrindah	85	34.0
4	22091397005	Furqon	88	35.2
5	22091397006	Hilal	83	33.2
6	22091397007	Ulung	88	35.2
7	22091397008	Yolanda	82	32.8
8	22091397009	Habibil	95	38.0
9	22091397010	Fajrul	93	37.2

Kita juga dapat menghitung rata-rata nilai mahasiswa dan menampilkan hasilnya menggunakan Pandas

```
# Menghitung rata-rata nilai 'Grade'
rata_rata_alpro = df['nilai_alpro'].mean()

# Menampilkan hasil rata-rata
print("Rata-rata nilai Alpro mahasiswa:",rata_rata_alpro)
```

Untuk mengelompokkan data berdasarkan kolom Status dan menghitung jumlah mahasiswa untuk setiap status, kita bisa menggunakan fungsi groupby() di Pandas. Misalkan kita ingin melihat jumlah mahasiswa berdasar perolehan nilai_alpro

```
kelompok = df.groupby('nilai_alpro').size()
print(kelompok)
```

Dengan output berikut

nilai_alpro	
82	1
83	1
85	2
88	2
90	1
93	1
95	2

2. Menulis File CSV dengan Pandas

Gunakan fungsi DataFrame.to_csv() untuk menyimpan DataFrame ke file CSV.

```
df.to_csv('data_baru.csv', index=False)
```

Setelah kode berhasil dijalankan maka akan terbuat file dengan nama data_baru.csv dengan isi sebagai berikut

data_baru.csv X

```
F: > UNESA > Bahan Ajar > Algoritma & Pemrograman > data_baru.csv
1  nim,nama,nilai_alpro,nilai_uas
2  22091397001, Asyifatul,90,36.0
3  22091397002, Roihan,95,38.0
4  22091397003, Insan,85,34.0
5  22091397004, Novrindah,85,34.0
6  22091397005, Furqon,88,35.2
7  22091397006, Hilal,83,33.2
8  22091397007, Ulung,88,35.2
9  22091397008, Yolanda,82,32.800000000000004
10 22091397009, Habibil,95,38.0
11 22091397010, Fajrul,93,37.2
```

6.2 Tugas Pratikum

1. Unduh file students.csv dengan isi sebagai berikut:
2. Baca file students.csv ke dalam DataFrame menggunakan Pandas.
3. Tampilkan 5 baris pertama dari DataFrame.
4. Tampilkan informasi tentang DataFrame menggunakan `info()` dan `describe()`.
5. Tambahkan kolom baru bernama Status dengan aturan:
 - Jika nilai Grade ≥ 80 , maka Status adalah "Lulus".
 - Jika tidak, maka Status adalah "Tidak Lulus".
6. Simpan DataFrame yang telah dimodifikasi ke dalam file baru bernama students_processed.csv. Pastikan file baru tersebut tidak menyimpan index (gunakan parameter `index=False`).
7. Hitung rata-rata nilai Grade mahasiswa dan tampilkan hasilnya.
8. Kelompokkan data berdasarkan Status dan hitung jumlah mahasiswa untuk setiap status.

BAB VII

MINI PROJECT

Tujuan Pembelajaran :

1. Mahasiswa mampu mengintegrasikan konsep dasar Python dalam proyek nyata.
2. Mahasiswa dapat merancang, mengimplementasikan, dan menguji solusi berbasis pemrograman.
3. Mahasiswa terbiasa berpikir logis dan sistematis dalam menyelesaikan permasalahan sederhana dengan Python.

7.1. Pengantar Mini Proyek

Mini proyek merupakan latihan akhir yang menggabungkan berbagai materi sebelumnya, termasuk penggunaan variabel, input/output, percabangan, perulangan, fungsi, dan struktur data. Proyek ini bertujuan agar mahasiswa dapat mengembangkan solusi sederhana berbasis teks menggunakan bahasa pemrograman Python.

7.2. Contoh Mini Proyek 1: Aplikasi Sistem Login Sederhana

Deskripsi: Program meminta pengguna memasukkan username dan password. Jika sesuai, ditampilkan pesan selamat datang. Jika tidak sesuai setelah 3 kali percobaan, akses ditolak.

Fitur:

- Validasi input
- Percabangan dan perulangan
- Struktur data dictionary

Contoh Kode:

```
Proyek 1.py

user_terdaftar = {"admin": "1234", "user": "abcd"}
percobaan = 0
maks_percobaan = 3

while percobaan < maks_percobaan:
    username = input("Masukkan username: ")
    password = input("Masukkan password: ")

    if username in user_terdaftar and user_terdaftar[username] == password:
        print("\nSelamat datang,", username)
        break
    else:
        print("\nUsername atau password salah.")
        percobaan += 1

if percobaan == maks_percobaan:
    print("\nAkses ditolak. Silakan coba lagi nanti.")
```

Hasil Output:

```
Masukkan username: admin
Masukkan password: 123456

Username atau password salah.
Masukkan username: admin
Masukkan password: 1234

Selamat datang, admin
```

7.3. Contoh Mini Proyek 2: Kalkulator Sederhana

Deskripsi: Program menerima dua angka dan operator dari pengguna, lalu menampilkan hasil perhitungan.

Fitur:

- Penggunaan fungsi
- Percabangan if-elif-else
- Validasi pembagian nol

Contoh Kode:

```
Proyek 2.py

def hitung(a, b, operator):
    if operator == "+":
        return a + b
    elif operator == "-":
        return a - b
    elif operator == "*":
        return a * b
    elif operator == "/":
        if b != 0:
            return a / b
        else:
            return "Error: Pembagian dengan nol"
    else:
        return "Operator tidak valid"

angka1 = float(input("Masukkan angka pertama: "))
angka2 = float(input("Masukkan angka kedua: "))
operator = input("Masukkan operator (+, -, *, /): ")

hasil = hitung(angka1, angka2, operator)
print("Hasil: ", hasil)
```

Hasil Output:

```
Masukkan angka pertama: 12
Masukkan angka kedua: 10
Masukkan operator (+, -, *, /): *
Hasil: 120.0
```

7.4. Contoh Mini Proyek 3: Manajemen Nilai Data Mahasiswa

Deskripsi: Program menyimpan data mahasiswa dan nilai, serta menampilkan rata-rata nilai tiap mahasiswa.

Fitur:

- Struktur data dictionary dan list
- Perulangan dan fungsi
- Menampilkan data secara sistematis

Contoh Kode:

```
Proyek 2.py

jumlah_mhs = int(input("Masukkan jumlah mahasiswa: "))
data_mahasiswa = {}

for i in range(jumlah_mhs):
    nama = input(f"\nMasukkan nama mahasiswa ke-{i+1}: ")
    nilai = []
    for j in range(3):
        n = float(input(f"  Masukkan nilai ke-{j+1}: "))
        nilai.append(n)
    data_mahasiswa[nama] = nilai

print("\nData Nilai Mahasiswa:")
for nama, nilai in data_mahasiswa.items():
    rata2 = sum(nilai) / len(nilai)
    print(f"{nama}: Nilai = {nilai}, Rata-rata = {rata2:.2f}")
```

Hasil Output:

```
Masukkan jumlah mahasiswa: 2

Masukkan nama mahasiswa ke-1: Deny
Masukkan nilai ke-1: 90
Masukkan nilai ke-2: 92
Masukkan nilai ke-3: 99

Masukkan nama mahasiswa ke-2: Pratama
Masukkan nilai ke-1: 91
Masukkan nilai ke-2: 92
Masukkan nilai ke-3: 93

Data Nilai Mahasiswa:
Deny: Nilai = [90.0, 92.0, 99.0], Rata-rata = 93.67
Pratama: Nilai = [91.0, 92.0, 93.0], Rata-rata = 92.00
```

7.5. Latihan Mini Proyek

1. Buatlah program "Aplikasi Konversi Suhu" yang meminta pengguna memilih konversi suhu (Celsius ke Fahrenheit, Fahrenheit ke Celsius, Celsius ke Kelvin).
2. Kembangkan program "Sistem Kasir Sederhana" yang mencatat daftar barang (nama dan harga), lalu menghitung total belanja dan kembalian.
3. Buat program "Pendaftaran Peserta Kegiatan" yang mencatat nama, usia, dan domisili peserta. Simpan dalam dictionary dan tampilkan daftar peserta dengan format tabel.

Profil Penulis



I Gde Agung Sri Sidhimantara, S.Kom., M.Kom.

Penulis dilahirkan di Singaraja, pada 26 Oktober 1995. Ia memulai pendidikan tinggi di bidang Teknik Informatika pada tahun 2013 di Institut Teknologi Sepuluh Nopember (ITS) Surabaya, dan melanjutkan pendidikan jenjang magister di bidang yang sama pada tahun 2018 di kampus yang sama. Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Algoritma dan Pemrograman, Struktur Data, Pemrograman Berbasis Framework, Pemrograman Mobile, Kecerdasan Buatan, Pemrograman Berorientasi Objek, dan Pemrograman Web.

Email Penulis : igdesidhimantra@unesa.ac.id



**Binti Kholifah, S.Kom.,
M.Tr.Kom.**

Penulis lahir di Nganjuk pada tanggal 06 Agustus 1996. Menempuh pendidikan tinggi jenjang sarjana di Universitas Islam Negeri (UIN) Maulana Malik Ibrahim Malang pada tahun 2014 dengan program studi Teknik Informatika. Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan magister di jurusan Teknik

Informatika dan Komputer pada tahun 2019 di Politeknik Elektronika Negeri Surabaya (PENS). Sebelum menjadi dosen tetap pada Program Studi D4 Manajemen Informatika, penulis memiliki pengalaman menjadi dosen di Institut Teknologi Mojosari (ITM) Nganjuk. Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Algoritma dan Pemrograman, Desain UI/UX, Matematika Diskrit, Pemrograman Game, Pemrograman API, Pemrograman Berorientasi Objek, dan Struktur Data.

Email Penulis : bintikholfah@unesa.ac.id



**M Adamu Islam Mashuri,
S.Tr.T., M.Tr.Kom**

Penulis lahir di Sidoarjo, pada 16 januari 1999. Menempuh pendidikan tinggi jenjang sarjana di jurusan Teknik Telekomunikasi pada tahun 2017 di Politeknik Elektronika Negeri Surabaya (PENS). Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan jenjang magister di jurusan Teknik Informatika dan

Komputer pada tahun 2021 di kampus yang sama. Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Algoritma dan Pemrograman, Struktur Data, Analisis Dan Disain Perangkat Lunak, Basis Data, Jaringan Komputer, Rekayasa Perangkat Lunak, dan Struktur Data.

Email Penulis : mmashuri@unesa.ac.id



**Faris Abdi El Hakim, S.Kom.,
M.Tr.Kom.**

Penulis lahir di Lamongan pada tanggal 16 Agustus 1995. Menempuh pendidikan tinggi jenjang sarjana di Universitas Brawijaya (UB) pada tahun 2013 dengan program studi Teknik Informatika. Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan magister jurusan Teknik

Informatika dan Komputer pada tahun 2021 di Politeknik Elektronika Negeri Surabaya (PENS). Sebelum menjadi dosen tetap pada Program Studi D4 Manajemen Informatika, penulis memiliki pengalaman profesional di perusahaan swasta sebagai Project Manager dan Programmer. Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Algoritma dan Pemrograman, Basis Data, Pemrograman Web, Pemrograman API, Struktur Data, Desain UI/UX, Pengantar Manajemen Proses Bisnis, dan Statistik.

Email Penulis : farishakim@unesa.ac.id



**Salamun Rohman Nudin,
S.Kom., M.Kom.**

Penulis lahir di Jombang pada tanggal 02 November 1982. Menempuh pendidikan tinggi jenjang sarjana di Universitas 17 Agustus 1945 Surabaya pada tahun 2004 dengan program studi Teknik Informatika. Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan magister jurusan Teknik

Informatika pada tahun 2009 di Institut Teknologi Sepuluh November (ITS). Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Analisis Big Data, Analisis Dan Disain Perangkat Lunak, Kecerdasan Buatan, Metodologi Penelitian, Sistem Informasi Manajemen, Web Cerdas dan Big Data.

Email Penulis : salamunrohman@unesa.ac.id



Asmunin, S.Kom., M.Kom.

Penulis lahir di Jombang pada tanggal 10 Januari 1977. Menempuh pendidikan tinggi jenjang sarjana di Institut Teknologi Sepuluh November (ITS) pada tahun 1997 dengan program studi Teknik Informatika. Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan magister jurusan Teknik Informatika pada tahun

2019 di Institut Teknologi Sepuluh November (ITS). Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Kecerdasan Buatan, Keamanan Perangkat Lunak, Sistem Informasi Manajemen, Sistem Informasi Manajemen, Rekayasa Perangkat Lunak, Probabilitas dan Statistika.

Email Penulis : asmunin@unesa.ac.id



**Andi Iwan Nurhidayat, S.Kom.,
M.T.**

Penulis lahir di Jember pada tanggal 27 Oktober 1978. Menempuh pendidikan tinggi jenjang sarjana di Institut Teknologi Sepuluh November (ITS) pada tahun 1998 dengan program studi Teknik Informatika. Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan magister jurusan

Jaringan Cerdas Multimedia pada tahun 2011 di Institut Teknologi Sepuluh November (ITS). Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Algoritma dan Pemrograman, Aljabar Linier dan Matrik, Analisis Dan Disain Perangkat Lunak, Basis Data, Basis Data Lanjut, Jaringan Komputer, Grafika Komputer dan Pemrograman API.

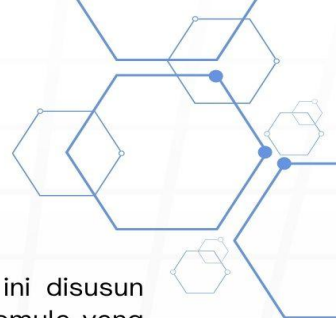
Email Penulis : andy134k5@unesa.ac.id



Ari Kurniawan, S.Kom., M.T.

Penulis lahir di Jombang pada tanggal 30 Maret 1973. Menempuh pendidikan tinggi jenjang sarjana di Universitas Dinamika (STIKOM) pada tahun 1998 dengan program studi Teknik Informatika. Setelah menyelesaikan studi sarjana, penulis melanjutkan pendidikan magister jurusan Telematika pada tahun 2008 di Institut Teknologi Sepuluh November (ITS). Penulis aktif di Program Studi D4 Manajemen Informatika, Fakultas Vokasi, Universitas Negeri Surabaya Mata kuliah yang diampu antara lain: Analisis Dan Disain Perangkat Lunak, Basis Data, Pemrograman Web, Sistem Informasi Manajemen dan Pemrograman Berorientasi Objek.

Email Penulis : arikurniawan@unesa.ac.id



Sinopsis Buku:

Buku Algoritma dan Pemrograman Python ini disusun sebagai panduan praktis bagi mahasiswa dan pemula yang ingin memahami konsep dasar pemrograman komputer menggunakan bahasa Python. Dengan pendekatan sistematis, buku ini membahas mulai dari pengenalan Python, penulisan kode, penggunaan variabel dan operator, hingga percabangan, perulangan, fungsi, dan pengolahan file.

Setiap bab dirancang untuk membangun pemahaman bertahap melalui penjelasan konsep, contoh kode, dan latihan yang aplikatif. Disertai dengan mini proyek di akhir buku, pembaca diajak menerapkan keterampilan pemrograman dalam situasi nyata seperti sistem login, kalkulator, hingga manajemen data mahasiswa.

Python dipilih karena sintaksnya yang sederhana dan fleksibel, cocok untuk pemula sekaligus relevan dalam dunia industri dan penelitian. Dengan buku ini, pembaca tidak hanya belajar menulis kode, tetapi juga mengasah logika berpikir algoritmik yang menjadi dasar dalam pemecahan masalah secara komputasional.

Fitur Utama Buku:

- Pengenalan Bahasa Python
- Variabel, Operator, Input
- Logical dan Conditional Operator
- Looping
- Fungsi
- File
- Mini Project

Kontak Penerbit:

Penerbit: S1 Terapan Manajemen Informatika, Fakultas
Vokasi, Universitas Negeri Surabaya
Website: <https://terapan-ti.vokasi.unesa.ac.id/>
Email: d4mi@unesa.ac.id